

Algoritma dan Struktur Data

“Bagaimanakah caranya kamu dapat merancang dan menulis instruksi/program yang membuat komputer dapat melakukan pemrosesan yang diperlukan untuk menyelesaikan permasalahan yang diberikan?”





Tujuan Pembelajaran

Setelah mempelajari bab ini, kamu akan mampu membuat solusi rancangan program sederhana dalam bentuk pseudokode dengan menerapkan struktur data standar dengan himpunan data berstruktur kompleks.

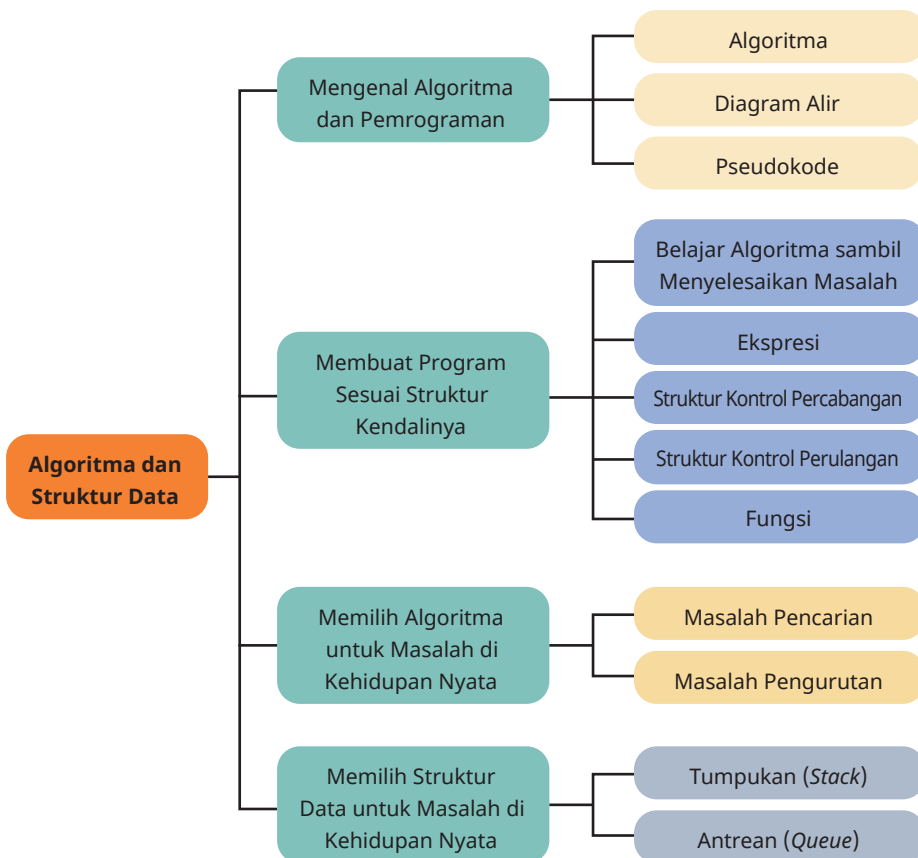


Kata Kunci

Algoritma, diagram alir, ekspresi, percabangan, perulangan.



Peta Konsep



Pada bab ini, kamu akan mengenal tentang algoritma dan struktur data. Dalam mata pelajaran Matematika, kamu telah mengenali adanya prosedur yang dapat digunakan untuk menghitung luas dan volume suatu bangun ruang dengan cara yang terstruktur dan menghasilkan keluaran yang benar. Dalam kehidupan, kamu menghasilkan dan mengolah banyak data, misalnya nilai-nilai di sekolah, pengeluaran belanja harianmu, sampai barang-barang yang kamu miliki. Agar dapat bermanfaat, data tersebut perlu disimpan dan diolah secara terstruktur. Pada bab ini, kamu akan mengenali konsep-konsep dasar algoritma dan struktur data yang dapat digunakan untuk menyimpan dan mengolah data tersebut menjadi suatu informasi yang bermanfaat bagi manusia. Dengan hal ini, kamu bahkan dapat membuat program komputer yang dapat melakukan proses tersebut secara cepat dan otomatis. Jangan khawatir jika kamu tidak memiliki komputer karena sebagian proses pemrograman tidak perlu menggunakan komputer.

Sebelum kamu bertualang dalam pembelajaran kali ini, jawablah dulu beberapa pertanyaan berikut.

1. Apa itu algoritma?
2. Unsur-unsur apa saja yang ada dalam sebuah algoritma?

A. Mengetahui Algoritma dan Pemrograman

Pada bagian ini, kamu akan mengenal lebih jauh tentang algoritma. Setelah itu, kamu akan belajar bagaimana diagram alir akan membentuk suatu algoritma. Akhirnya, kamu akan mengenal pseudocode, kode semu yang akan melengkapi kekurangan pada suatu algoritma.

1. Algoritma

Algoritma adalah suatu kumpulan instruksi terstruktur dan terbatas yang dapat diimplementasikan di dalam kehidupan, termasuk dalam bentuk program komputer untuk menyelesaikan suatu permasalahan tertentu. Algoritma juga merupakan bentuk dari suatu strategi atau ‘resep’ yang kamu gunakan untuk menyelesaikan suatu masalah. Algoritma lahir dari suatu proses berpikir komputasional oleh seseorang untuk menemukan solusi dari suatu permasalahan yang diberikan. Dengan demikian, berpikir komputasional merupakan keahlian yang kamu perlukan untuk dapat membuat algoritma, program, atau suatu karya informatika yang dapat digunakan dengan efektif dan efisien.



Kamu menganalisis suatu problem menggunakan teknik abstraksi dan dekomposisi, kemudian menyusun algoritma dengan melakukan pengenalan pola dari problem sejenis. Algoritma tersebut harus direpresentasikan dalam bentuk yang dapat dipahami oleh orang lain. Selain itu, karena pada akhirnya akan diubah dalam bentuk kode program, algoritma tersebut harus ditulis dalam bentuk yang terdefinisi dengan baik (*well-defined*) dengan jumlah langkah yang terbatas. Algoritma merupakan rancangan logis dan sistematis dari sebuah program. Dengan demikian, kemampuan menuliskan algoritma dengan baik akan membantu kamu dalam membuat program yang baik dan benar.

Algoritma tidak harus selalu terkait dengan pemrograman. Kamu sebenarnya sering menemui algoritma dalam kehidupan sehari-hari. Pada dasarnya, sebuah rangkaian instruksi yang tersusun secara sistematis dan digunakan untuk menyelesaikan suatu permasalahan atau mencapai tujuan tertentu, dapat disebut sebagai algoritma. Misalnya, sebuah resep bagaimana memasak suatu hidangan dapat disebut sebagai sebuah algoritma.

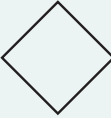
Pada bagian ini, kamu akan mempelajari dua cara untuk merepresentasikan algoritma, yaitu lewat diagram alir dan pseudokode. Untuk itu, kamu perlu mempelajari teknik untuk membaca suatu algoritma (yang disebut sebagai penelusuran atau *tracing*) dan cara untuk menuliskan suatu algoritma. Perlu diingat bahwa menulis algoritma berbeda dengan menulis program. Program ditulis agar dapat dipahami oleh mesin, sedangkan algoritma ditulis agar dapat dipahami oleh manusia. Untuk program yang sederhana, algoritma akan sangat mirip, bahkan sama dengan program. Jika persoalan makin kompleks, algoritma hanya berisi abstraksi, yang akan mempermudah implementasinya menjadi program.

2. Diagram Alir

Diagram alir merupakan notasi algoritma dengan menggunakan gambar atau simbol sebagai penjelas dari suatu algoritma. Diagram alir memungkinkan suatu proses dapat dipahami dengan mudah dan cepat. Diagram alir dibuat dalam bentuk aliran simbol yang dapat ditelusuri dari suatu titik permulaan hingga titik akhir dari program. Diagram alir dibuat menggunakan simbol standar ANSI/ISO yang beberapa simbol dasarnya diberikan pada Tabel 2.1.



Tabel 2.1 Simbol Diagram Alir Beserta Maknanya

Simbol	Nama	Deskripsi
	Garis alir (<i>flowline</i>)	Arah yang menunjukkan aliran program dari awal hingga akhir.
	Terminator	Titik awal atau titik akhir suatu program.
	Proses	Suatu kegiatan komputasi yang dilakukan oleh program: misalnya operasi aritmatika.
	Percabangan	Merupakan titik percabangan yang salah satu cabangnya dapat dilalui oleh program berdasarkan suatu kondisi.
	Masukan (<i>Input</i>)/ Keluaran (<i>Output</i>)	Melambangkan titik saat program akan menerima suatu data atau menghasilkan suatu informasi.
	Subprogram	Melambangkan suatu kegiatan atau proses lain yang telah didefinisikan sebelumnya.
	Penghubung dalam Halaman	Digunakan untuk menghubungkan suatu titik pada diagram alir ke titik lain pada halaman yang sama.
	Penghubung Antarhalaman	Digunakan untuk menghubungkan suatu titik pada diagram alir ke titik lain pada halaman yang berbeda. Digunakan apabila diagram lain cukup kompleks sehingga tidak dapat digambar dalam satu halaman.

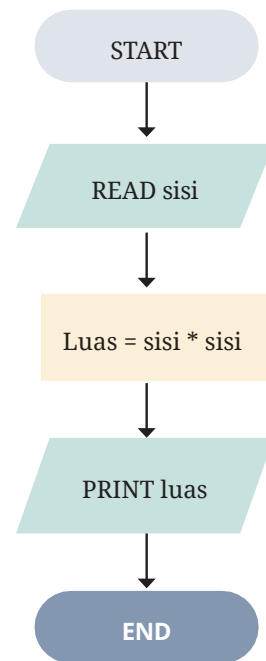
Untuk memahami bagaimana diagram alir digunakan untuk menggambarkan suatu algoritma, pada bagian berikut, diberikan lima buah contoh diagram alir dari beberapa proses berpikir yang telah kamu kenal.



a. Menghitung Luas Persegi

Diagram alir di samping dapat dibaca sebagai berikut.

- Diagram alir dibaca mulai dari simbol START, lalu mengikuti arah panah.
- Untuk menghitung luas persegi, kamu memerlukan sebuah data, yaitu panjang sisi. Panjang sisi ini dibaca pada diagram alir dengan menggunakan kata kunci READ dan disimpan dalam sebuah variabel bernama *sisi*.
- Setelah itu, kamu melakukan suatu proses ekspresi matematika untuk menghitung luas persegi menggunakan rumus yang telah kamu ketahui, yaitu $\text{luas} = \text{sisi} \times \text{sisi}$. Hasil perhitungan tersebut disimpan pada sebuah variabel bernama *luas*.
- Walaupun telah memperoleh jawaban yang dicari, komputer perlu diinstruksikan secara spesifik untuk mengeluarkan jawaban tersebut.
- Setelah PRINT, algoritma berakhir karena simbol setelahnya ialah simbol END.



Secara ringkas, kamu pun dapat menulis diagram alir tersebut dalam tiga bagian, yaitu Masukan (*Input*), Proses, dan Luaran (*Output*) menjadi:

INPUT/MASUKAN : sisi

PROSES : $\text{luas} = \text{sisi} \times \text{sisi}$

OUTPUT/LUARAN : luas

b. Menghitung Luas Permukaan Kubus

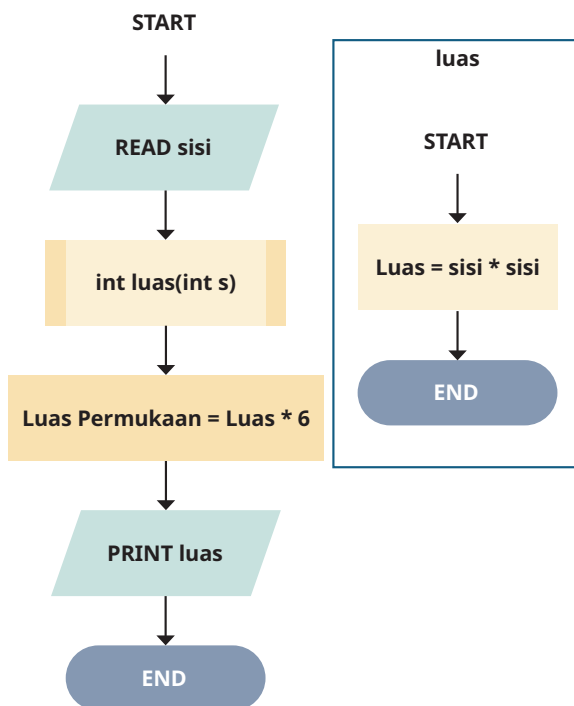
Saat menyusun solusi untuk menyelesaikan suatu permasalahan, kamu sering kali membutuhkan solusi dari permasalahan lain yang lebih sederhana. Misalnya, kita harus menghitung luas permukaan dari sebuah kubus. Pada prosesnya, kamu perlu menghitung luas persegi yang membentuk kubus tersebut. Hal ini dapat digambarkan pada diagram alir menggunakan simbol subprogram. Pada diagram alir ini, terlihat bahwa



proses akan memanggil subprogram menghitung luas persegi yang telah kamu buat sebelumnya. Subprogram dapat kamu gunakan untuk menggambarkan abstraksi dan dekomposisi yang telah kamu pelajari pada berpikir komputasional.

Diagram alir utama di samping dapat ditulis sebagai:
 INPUT/MASUKAN : sisi
 PROSES : luas * 6
 OUTPUT/LUARAN : luas permukaan

Diagram alir dari subprogram dapat ditulis sebagai:
 INPUT/MASUKAN : sisi
 PROSES : luas = sisi * sisi
 OUTPUT/LUARAN : luas



c. Membagi Bilangan

Diagram alir dapat memiliki beberapa kemungkinan aliran sehingga suatu algoritma dapat adaptif terhadap masukan yang diberikan. Hal ini dimungkinkan dengan adanya simbol percabangan. Aliran keluar dari simbol percabangan akan bergantung pada kondisi yang ada di dalam simbol percabangan.

Pada contoh ini, simbol percabangan digunakan untuk menghindari dijalankannya suatu operasi matematika yang tidak dapat dieksekusi oleh komputer, yaitu operasi pembagian dengan pembagi bernilai 0. Apabila operasi tersebut dilakukan, komputer akan menampilkan pesan kesalahan dan program akan berhenti secara tidak wajar.



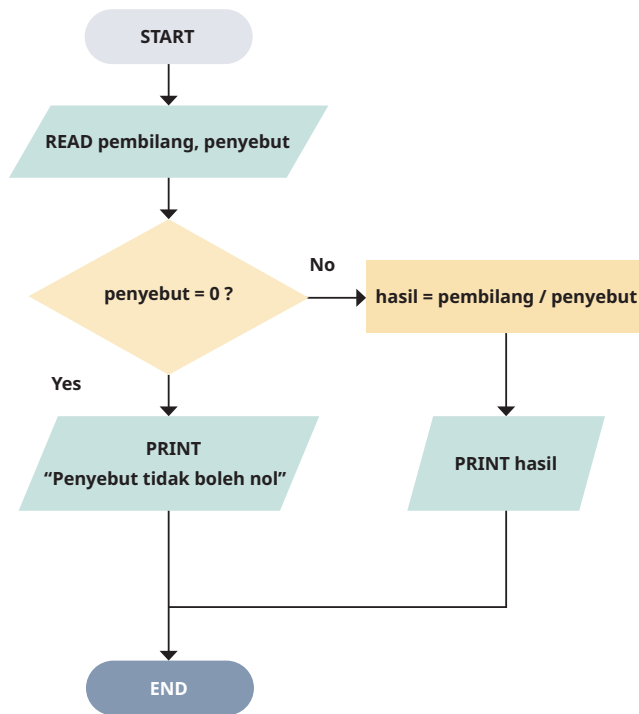


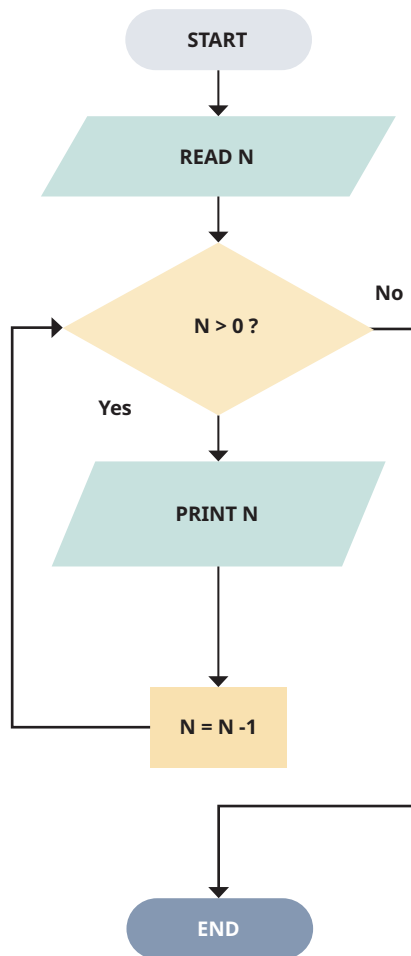
Diagram alir ini merupakan proses untuk membagi *pembilang* dengan *penyebut*. Akan tetapi, sebelum operasi pembagian dilakukan, diagram akan mengecek terlebih dahulu nilai dari *penyebut*. Apabila *penyebut* bernilai 0, operasi pembagian tidak dilakukan dan pesan yang sesuai akan ditampilkan. Jika tidak, operasi dapat dilakukan dengan aman dan hasil pembagian dapat ditampilkan.

d. Menghitung Mundur dari N hingga 1

Aliran pada diagram alir dapat diatur sehingga satu lebih simbol dijalankan berulang kali. Pada contoh berikut, perulangan dilakukan sehingga diagram alir tersebut akan menghasilkan barisan bilangan bulat dari N hingga 1. Misalkan N bernilai 5, diagram alir akan mencetak angka 5 4 3 2 1.

Tentunya, perulangan tidak dapat dilakukan terus-menerus sehingga diperlukan suatu kondisi untuk menghentikan perulangan. Simbol percabangan dapat digunakan untuk menghentikan perulangan tersebut pada kondisi yang kita tetapkan.

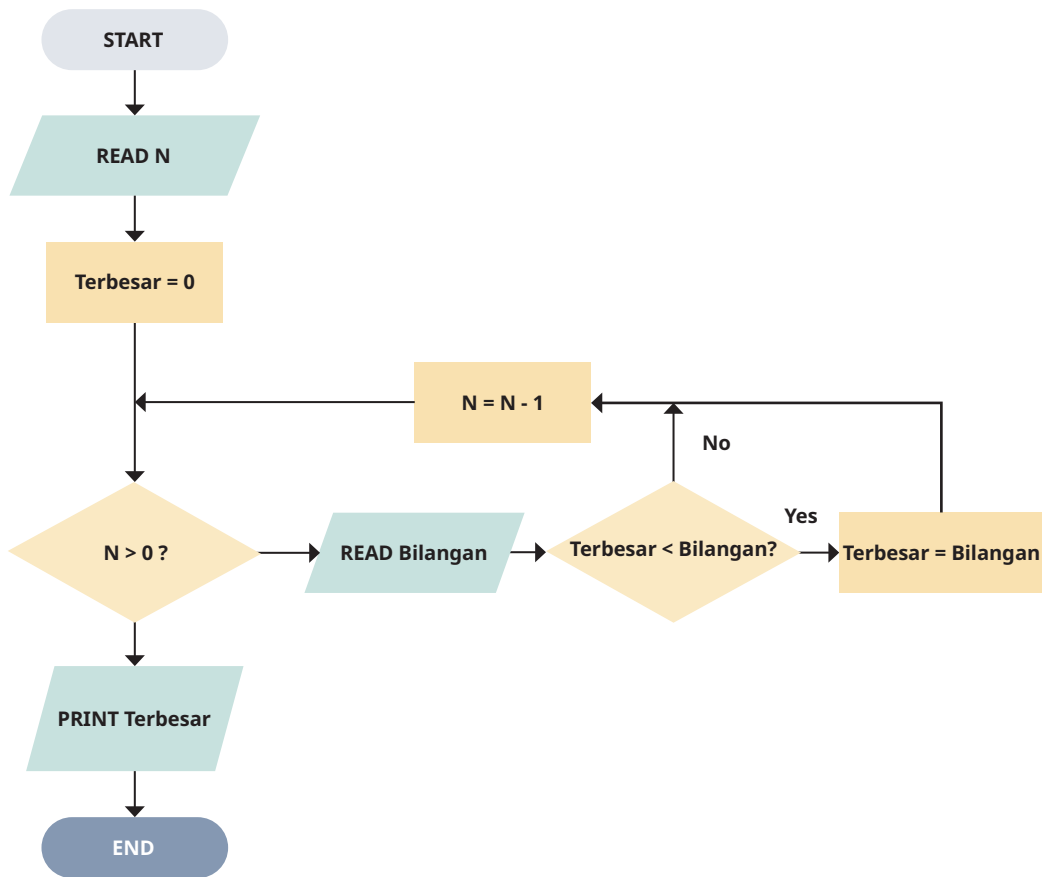




e. Mencari Bilangan Terbesar dari Suatu Himpunan Bilangan

Tentunya, simbol-simbol dasar pada diagram alir dapat dipadukan untuk menghasilkan sebuah proses yang lebih kompleks. Diagram alir berikut menggambarkan proses mencari bilangan terbesar dari suatu himpunan N buah bilangan. Diagram alir berikut akan membaca sebanyak N buah bilangan dan akan menghasilkan bilangan yang paling besar di antara bilangan tersebut.





f. Menelusuri Diagram Alir

Di buku kerjamu, kamu dapat melakukan penelusuran (*tracing*) secara terstruktur dengan membuat tabel sederhana yang terbagi tiga bagian, yaitu masukan, nilai variabel, dan keluaran. Bagian masukan akan diisi dengan data yang akan diproses, bagian nilai variabel akan menjadi tempatmu mencatat nilai yang disimpan di dalam variabel, dan bagian keluaran akan diisi dengan data yang dihasilkan oleh diagram alir. Penelusuran ini menjadi penting saat kamu ingin memahami perilaku dari suatu algoritma, atau saat kamu ingin mengecek ketepatan keluaran dari suatu algoritma.

Sebagai contoh, misal kamu mencari bilangan terbesar dari 4 bilangan berikut: 1, 3, 2, 4 menggunakan diagram alir kelima. Ada beberapa kegiatan inti yang akan kamu lakukan dalam melakukan penelusuran.



- Mempersiapkan lembar kerja penelusuran (Tabel 2.2.a), menuliskan data yang akan diolah, menuliskan nama variabel yang digunakan pada diagram alir, dan mengosongkan bagian keluaran.
- Menelusuri diagram alir dari bagian permulaan.
- Ketika menemukan simbol untuk membaca suatu data, kamu dapat mencoret masukan yang dibaca, kemudian meletakkannya ke variabel yang tepat. Misalnya, berada pada simbol masukan READ N . Letakkan bilangan pertama di bagian masukan (yaitu 4) ke variabel N (Tabel 2.2.b).
- Ketika tiba di suatu simbol proses yang menyimpan suatu nilai pada variabel (penugasan atau *assignment*), meletakkan nilai tersebut pada bagian nilai variabel. Misalnya, saat bertemu dengan $Terbesar = 0$ pada diagram alir, menuliskan nilai 0 pada variabel terbesar (Tabel 2.2.c).
- Proses juga dapat berisi ekspresi matematika, misalnya $N = N - 1$. Untuk mengerjakan ekspresi tersebut, kerjakan dahulu bagian kanan dari ekspresi, yaitu $N - 1$. Cek nilai N saat ini di bagian nilai variabel, dan akan menemukan nilai 4. Kerjakan ekspresi tersebut, yaitu $4 - 1 = 3$, lalu simpan hasilnya ke sisi kanan dari ekspresi, yaitu variabel N . Lewat ekspresi ini, nilai N yang tadinya 4, sekarang telah berubah menjadi 3. Kamu dapat mencoret nilai 4 dan menuliskan nilai 3 pada Lembar Kerja (Tabel 2.2.d).
- Kemudian, dapat melanjutkan proses dan akhirnya menemukan nilai N sekarang bernilai 0. Lembar kerjamu akan berisi seperti Tabel 2.2.e.
- Kemudian, menemukan simbol keluaran PRINT Terbesar. Pada tahap ini, kamu dapat menuliskan isi dari variabel terbesar ke dalam bagian keluaran di lembar kerjamu (Tabel 2.2.f).
- Terakhir, menelusuri dan menemukan terminator END sehingga penelusuran berakhir. Dengan demikian, diagram alir tadi menghasilkan keluaran berupa nilai 4 pada kasus yang diberikan. *Selamat! Kamu telah berhasil menelusuri diagram alir!*



Tabel 2.2 Ilustrasi Penelusuran Diagram Alir di Buku Kerja Siswa

(a) Sebelum menelusuri.	(b) Saat menemukan simbol masukan dengan perintah READ N .	(c) Saat menemukan simbol proses dengan perintah terbesar = 0.
Masukan 4 1 3 2 4	Masukan 4 1 3 2 4	Masukan 4 1 3 2 4
Nilai variabel N: Bilangan: Terbesar:	Nilai variabel N : 4 Bilangan: Terbesar:	Nilai variabel N : 4 Bilangan: Terbesar: 0
Keluaran	Keluaran	Keluaran
(d) Saat menemukan simbol proses dengan ekspresi $N = N - 1$.	(e) Saat N bernilai 0, Lembar Kerja akan menjadi seperti ini.	(f) Saat menemukan simbol keluaran PRINT Terbesar.
Masukan 4 1 3 2 4	Masukan 4 1324	Masukan 4 1324
Nilai variabel N : 4 3 Bilangan: Terbesar:	Nilai variabel N : 4 324 0 Bilangan: 1 324 0 Terbesar: 4	Nilai variabel N : 4 324 0 Bilangan: 1 324 0 Terbesar: 4
Keluaran	Keluaran	Keluaran 4

Walaupun ilustrasi ini terdiri atas beberapa tabel, kamu cukup bekerja dengan satu tabel saat menelusuri diagram alir. Teknik penelusuran ini tidak hanya dapat digunakan untuk membaca suatu diagram alir, tetapi



juga dapat digunakan untuk membaca pseudokode atau kode program. Untuk algoritma yang pendek, kamu mungkin tidak membutuhkan lembar kerja seperti ini, tetapi lembar kerja ini akan sangat bermanfaat ketika menelusuri suatu algoritma yang panjang dan kompleks. Sekarang, saatnya kamu berlatih menggunakan teknik ini!



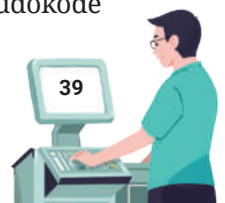
Aktivitas 10.2-01-U Menelusuri Diagram Alir

Sebelum membuat diagram alir sendiri, kamu perlu kemampuan untuk membaca diagram alir yang telah tersedia. Oleh karena itu, pada latihan ini, kamu akan menelusuri diagram alir yang telah tersedia di atas ketika diberikan suatu kasus untuk diselesaikan. Walaupun kamu telah mengetahui apa hasilnya, lakukanlah penelusuran secara terstruktur mulai dari awal hingga akhir diagram alir.

1. Hitunglah luas persegi dengan panjang sisi 3 cm menggunakan diagram alir pertama.
2. Hitunglah luas permukaan sebuah kubus yang memiliki panjang sisi 20 cm dengan menggunakan diagram alir kedua.
3. Hitunglah dua buah operasi pembagian berikut dengan menggunakan diagram alir ketiga. Pertama, pembagi bernilai 10 dan penyebut bernilai 2. Kedua, pembagi bernilai 8 dan penyebut bernilai 0.
4. Lakukan hitung mundur dari angka 5 hingga 1 dengan menelusuri diagram alir keempat.

3. Pseudokode

Pseudokode (kode semu atau kode pseudo) adalah suatu bahasa buatan manusia yang sifatnya informal untuk merepresentasikan algoritma menggunakan konvensi gabungan bahasa pemrograman. Pseudokode dibuat untuk menutupi kekurangan diagram alir dalam merepresentasikan konsep-konsep pemrograman terstruktur. Caranya dengan memberikan representasi langkah-langkah yang lebih detail dan dekat dengan bahasa pemrograman. Pseudokode



juga dapat merepresentasikan penggunaan simbol aritmatika, logika, dan penugasan (*assignment*) dengan lebih baik. Karena sifatnya yang informal, tidak ada aturan khusus dalam standar notasi yang dapat digunakan. Akan tetapi, ada beberapa prinsip dasar yang perlu diperhatikan, yaitu satu baris untuk satu **pernyataan** (*statement*) dan pentingnya indentasi dalam menuliskan pernyataan. Indentasi ada untuk hierarki dari pernyataan. Misalnya, untuk menunjukkan bahwa suatu pernyataan merupakan bagian dari sebuah struktur kontrol percabangan atau struktur kontrol perulangan (lihat konsep blok pada pemrograman visual yang telah kamu pelajari di tingkat SMP).

Kelima diagram alir pada bagian sebelumnya dapat ditulis dalam bentuk pseudokode sebagai berikut.

Tabel 2.3 Pseudokode 1: Menghitung Luas Persegi

Deskripsi Naratif	Pseudokode
1. Baca nilai sisi persegi. 2. Hitung luas dengan mengkuadratkan nilai sisinya. 3. Cetak luas.	Algoritma Menghitung Luas Persegi Input: Nilai Panjang Sisi Persegi. Proses: BACA sisi luas $\leftarrow$ sisi * sisi CETAK luas Output: Luas Persegi tercetak.

Tabel 2.4 Pseudokode 2: Menghitung Luas Permukaan Kubus

Deskripsi Naratif	Pseudokode
1. Baca nilai sisi kubus. 2. Hitung luas (persegi) dari sisi kubus. Kalikan luas kubus dengan angka 6 (banyaknya jumlah persegi pada kubus) untuk mendapatkan luas permukaan. 3. Cetak luas_permukaan.	Algoritma Menghitung Luas Permukaan Kubus Input: Nilai Panjang Sisi Kubus. Proses: BACA sisi luas_permukaan $\leftarrow$ luas(sisi) * 6 CETAK luas_permukaan Output: Luas Permukaan Kubus tercetak.



Tabel 2.5 Pseudokode 3: Membagi Bilangan

Deskripsi Naratif	Pseudokode
1. Baca nilai pembilang dan penyebut. 2. Jika penyebut bernilai 0, cetak tulisan “Penyebut tidak boleh nol”. 3. Jika penyebut tidak bernilai 0, lakukan pembagian pembilang dengan penyebut dan simpan hasilnya. 4. Cetak hasil pembagian.	Algoritma Membagi Bilangan Input: Pembilang dan Penyebut. Proses: BACA pembilang, penyebut IF penyebut = 0 PRINT “Penyebut tidak boleh nol” ELSE hasil $\leftarrow$ pembilang / penyebut CETAK hasil Output: Hasil Pembagian tercetak.

Tabel 2.6 Pseudokode 4: Menghitung Mundur dari N hingga 1

Deskripsi Naratif	Pseudokode
1. Baca nilai N. 2. Selama $N > 0$ Ulangi. 3. Cetak tulisan N. 4. Kurangi nilai N dengan 1.	Algoritma Menghitung Mundur dari N hingga 1 Input: Nilai N. Proses: BACA N WHILE $N > 0$ CETAK N $N \leftarrow N - 1$ Output: Angka hasil hitung mundur dari nilai N sampai 1 tercetak.



Tabel 2.7 Pseudokode 5: Mencari Bilangan Terbesar dari Suatu Himpunan Bilangan

Deskripsi Naratif	Pseudokode
- Jika himpunan bilangan kosong, tidak ada bilangan terbesar. - Jika himpunan bilangan tidak kosong, asumsikan bilangan pertama sebagai bilang terbesar saat ini. - Untuk setiap bilangan anggota himpunan: bandingkan bilangan tersebut dengan bilangan terbesar saat ini. Apabila bilangan tersebut lebih besar, bilangan tersebut akan menjadi bilangan terbesar saat ini. - Apabila langkah 3 telah dilakukan pada semua bilangan, bilangan terbesar saat ini akan menjadi bilangan terbesar di himpunan bilangan tersebut.	Algoritma Mencari Bilangan Terbesar Input: Himpunan bilangan L. Proses: IF size of (L)= 0 RETURN null largest ← L[0] FOR EACH item in L, DO IF item > largest, THEN largest ← item RETURN largest Output: Bilangan terbesar pada himpunan bilangan L

Perhatikan bahwa pseudokode mengandung kata-kata yang pada contoh di atas, dicetak dengan huruf kapital seperti IF, THEN, FOR, RETURN, WHILE. Kata-kata tersebut merupakan kata-kata terbatas (*reserved word*) yang merupakan konvensi umum pada saat menulis bahasa pemrograman. Arti dari kata-kata tersebut secara lebih detail akan dijelaskan pada bab ini.

Algoritma yang telah disusun kemudian dapat dibuat menjadi suatu program komputer dengan melewati suatu proses pemrograman. Seorang pemrogram akan menggunakan bahasa pemrograman untuk menulis kode program yang akan dijalankan oleh komputer. Ada banyak bahasa pemrograman yang dapat digunakan dengan karakteristik masing-masing, misalnya C, C++, C#, Python, Java, dan R.

Aktivitas 10.2-02-U
Menelusuri Pseudokode

Dengan menggunakan instruksi yang sama dengan Aktivitas 10.2-01-U Menelusuri Diagram Alir, lakukan penelusuran pada pseudokode yang telah diberikan pada contoh.



Aktivitas 10.2-03-U **Menulis Algoritma**

Pada latihan ini, kamu diminta untuk menuliskan suatu algoritma berdasarkan deskripsi yang diberikan. Deskripsi ini berupa pernyataan dalam bahasa sehari-hari dari algoritma yang perlu kamu buat dalam bentuk diagram alir dan pseudokode. Setelah selesai, kamu dapat menunjukkan hasil pekerjaanmu kepada temanmu untuk ditelusuri.

Soal 1: Membayar Bakso (Tingkat Kesulitan: ★ ★)

Buatlah sebuah diagram alir atau pseudokode dari proses berikut.

Sebuah mesin pembayaran otomatis dirancang untuk mampu menangani pembayaran pembelian bakso secara mandiri. Mesin ini mampu untuk memberikan kembalian dalam bentuk uang kertas atau uang logam. Mesin akan menerima dua buah masukan, yaitu total bayar dan jumlah uang yang dibayarkan oleh pelanggan. Apabila jumlah uang yang dibayarkan lebih besar atau sama dengan total bayar, mesin akan menghitung kembalian yang harus diberikan kepada pelanggan. Apabila terjadi sebaliknya, mesin akan menampilkan teks “Uang yang dibayarkan kurang”.

Setelah diagram alir selesai, kamu dapat menelusurinya dengan menggunakan kasus berikut.

Kasus	Masukan	Keluaran
1	Total Bayar: 10.000 Jumlah Uang: 15.000	5.000
2	Total Bayar: 20.000 Jumlah Uang: 10.000	Uang yang dibayarkan kurang.

Soal 2: Hadiah Bakso Gratis (Tingkat Kesulitan: ★ ★ ★)

Kamu ialah pengusaha bakso yang sukses. Agar usaha baksomu bisa lebih berkembang, kamu berencana untuk menambah sentuhan teknologi sehingga beberapa proses dapat berjalan secara otomatis. Inovasi yang kamu pikirkan ialah menggunakan sistem poin untuk memberikan diskon kepada pelanggan. Poin ini akan diberikan pada saat pelanggan membayar di mesin pembayaran yang akan kamu



buat. Setiap membayar, pelanggan akan menerima poin senilai harga bakso yang dia beli. Apabila total poin mencapai 100.000, pelanggan akan menerima satu porsi bakso gratis.

Kamu lalu memikirkan suatu proses berikut: setelah memesan bakso, pelanggan dapat membayar dengan menggunakan ponsel miliknya. Kemudian, mesin tersebut menambahkan total pembayaran ke total poin yang saat ini dimiliki oleh pelanggan. Apabila total poin yang dimiliki pelanggan lebih besar dari 100.000, mesin akan mengeluarkan kalimat “Anda mendapatkan kupon bakso gratis” dan mengurangi total poin pelanggan dengan nilai 100.000. Setelah itu, mesin akan menampilkan total poin pelanggan saat ini.

Setelah diagram alir selesai, kamu dapat menelusurinya dengan menggunakan kasus berikut.

Kasus	Masukan	Keluaran
1	Total Pembayaran: 80.000 Total Poin Pelanggan Saat Ini: 10.000	Poin Anda saat ini: 90.000
2	Total Pembayaran: 20.000 Total Poin Pelanggan Saat Ini: 90.000	Anda mendapatkan kupon bakso gratis! Poin Anda saat ini: 10.000

Ayo, Renungkan!

Jawablah pertanyaan berikut dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal.

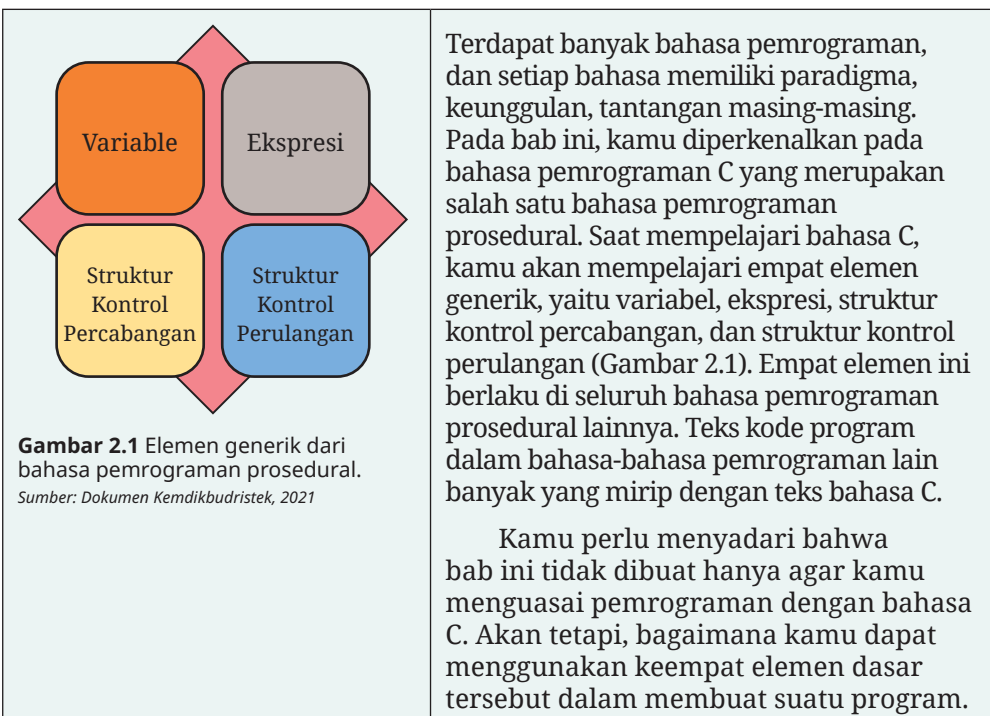
1. Apakah kamu merasa bahwa membuat algoritma dapat mempermudah kamu dalam menyelesaikan masalah?
2. Mana yang paling mudah untuk kamu gunakan, diagram alir atau pseudokode?
3. Setelah ini, apa yang akan kamu lakukan untuk dapat membaca dan menulis algoritma dengan baik?
4. Pelajaran paling berkesan apa yang kamu dapatkan dari pertemuan ini?



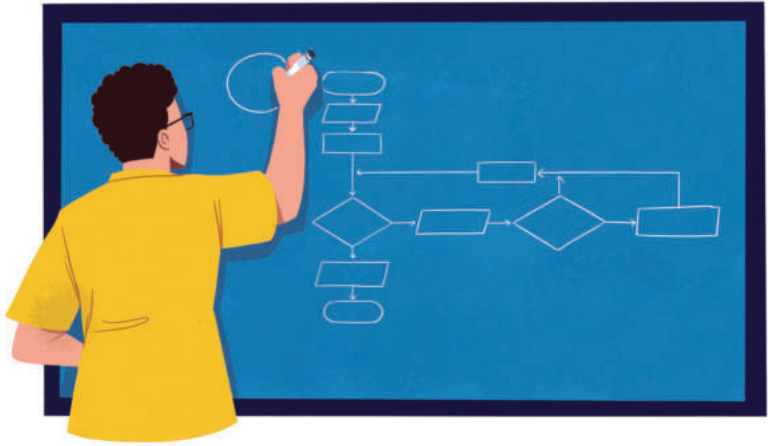
B. Membuat Program Sesuai Struktur Kendalinya

Belajar bahasa pemrograman sama halnya dengan belajar bahasa apa pun, dimulai dengan secara intuitif mengenal dan langsung memakai bahasa tersebut untuk keperluan sehari-hari yang penting sesuai kebutuhan, bukan dari teori bahasa. Seseorang dengan bahasa ibu bahasa Indonesia, saat belajar bahasa Inggris, akan mulai mengenal bahasa Inggris melalui “membaca” contoh-contoh kalimat sederhana, yang sangat diperlukan dalam kehidupan sehari-hari, seperti mulai menyapa “Selamat pagi”, “Jam berapa?”. Melalui contoh tersebut, dia akan belajar kosakata penting dan pola kalimat, misalnya kalimat pernyataan SPOK (Subjek, Predikat, Objek, Keterangan); struktur kalimat tanya, kalimat aklamasi, dan lain-lain. Selanjutnya, baru beranjak ke konsep yang lebih kompleks yang ada pada bahasa asing tersebut, dan mengenal tata bahasa secara lebih formal dan mulai menulis.

Belajar pemrograman pada hakikatnya sama dengan belajar bahasa natural (bahasa manusia sehari-hari): seseorang belajar dari “membaca” program terlebih dulu, daripada “menulis” kode program. Proses menulis kode (*coding*) dapat dimulai setelah kamu membaca contoh-contoh program yang menjadi pola pembangun program kompleks. Bedanya dengan belajar bahasa natural, teks dalam bahasa pemrograman yang ditulis bukan dilafalkan dan dipahami sebagai teks “statis”, melainkan juga dapat dipahami oleh mesin dan dapat dieksekusi (dijalankan).



1. Belajar Algoritma sambil Menyelesaikan Masalah



Aktivitas Ayo, Kita Berlatih sebagai bentuk latihan menyelesaikan suatu problem yang diberikan dengan pemrograman. Karena materi ini bersifat pengenalan pada kegiatan menulis kode program (*coding*), problem yang diberikan dalam bentuk spesifikasi yang telah terstruktur. Pada kenyataannya, ketika kamu membuat program untuk menyelesaikan suatu permasalahan nyata, kamu perlu membuat sendiri spesifikasi dari program yang akan dibuat. Kamu perlu menetapkan sendiri tujuan program, format masukan, serta format keluaran program. Aktivitas ini akan kamu lakukan di akhir bab. Sekarang, kamu dapat fokus membuat program berdasarkan spesifikasi yang telah diberikan.

Pada setiap aktivitas, kamu akan diberikan setidaknya satu permasalahan untuk diselesaikan dengan menggunakan konsep yang telah diberikan pada buku. Bentuk permasalahan tersebut akan diberikan dengan struktur pada Tabel 2.8.

Tabel 2.8 Spesifikasi Program yang Diberikan pada Problem Latihan

Bagian	Penjelasan
Nomor dan Nama Problem	Identitas dari permasalahan.
Deskripsi Soal	Memberikan konteks permasalahan yang perlu diketahui oleh siswa dalam membuat program.



Bagian	Penjelasan
Format Masukan	Memberikan susunan data yang diberikan pada program oleh pengguna, beserta ukuran dari data tersebut.
Format Keluaran	Memberikan susunan informasi yang akan dikeluarkan oleh program kepada pengguna.
Contoh Masukan	Memberikan contoh data yang dimasukkan.
Contoh Keluaran	Memberikan contoh informasi yang dikeluarkan program berdasarkan data yang dimasukkan.

Bagian format keluaran dan format masukan menjadi penting karena *autograder* (jika kamu menggunakan ini di kelas) akan memeriksa kode programmu dengan sangat ketat. Pemeriksaan tersebut berdasarkan pasangan masukan-keluaran yang telah disiapkan, atau disebut sebagai kasus uji (*test case*). Apabila program kamu membaca atau menulis dengan format yang salah, *auto-grader* tidak akan menganggap programmu benar. Untuk membantu kamu memahami format tersebut, satu atau lebih contoh masukan dan keluaran akan diberikan.

Praktik Baik Pemrograman

Pada setiap problem, kamu akan menemukan beberapa kasus uji yang dapat digunakan untuk menguji program yang kamu buat. Akan tetapi, bukan berarti kamu hanya perlu mengecek programmu dengan kasus uji yang ada di buku. Kamu perlu menguji programmu dengan menggunakan berbagai kasus uji yang kamu buat sendiri. Beberapa problem dirancang dengan adanya jebakan yang dapat membuat programmu gagal berjalan pada kasus tertentu. Misalnya, ketika masukan yang diberikan memiliki rentang yang besar atau adanya kondisi yang dapat menyebabkan program berhenti.

2. Ekspresi

Ekspresi merupakan bagian yang tidak terpisahkan dari program. Di dalam matematika, ekspresi terdiri atas kombinasi beberapa *operand* dan operator yang memiliki makna. Kamu telah terbiasa menulis ekspresi pada matematika, misalnya dalam penjumlahan $10 + 5$ yang melibatkan dua buah *operand* (10 dan 5) dan sebuah operator (+). Ekspresi pada pemrograman mirip dengan



ekspresi yang kamu pelajari pada matematika, tetapi diperkaya dengan tambahan operator untuk memudahkan kamu dalam menulis program.

a. Ekspresi berdasarkan Jumlah *Operand*

Berdasarkan jumlah *operand*, suatu ekspresi dapat dibagi menjadi 3 ekspresi berikut.

- *Unary* (satu *operand*), misalnya $-a$ untuk menegasikan suatu bilangan.
- *Binary* (dua *operand*), misalnya $a+b$ untuk penjumlahan.
- *Ternary* (tiga *operand*), misalnya $a ? b : c$ yang akan dijelaskan lebih rinci.

b. Operator berdasarkan Fungsi

Berdasarkan fungsinya, operator dapat dibagi menjadi operator penugasan, operator aritmatika, operator logika, serta *increment* dan *decrement*.

1) Operator Penugasan

Operator penugasan (simbol ‘=’) digunakan untuk memberikan suatu nilai konstanta atau nilai yang diperoleh dari suatu ekspresi ke dalam variabel. *Operand* di sisi kiri akan menerima nilai dari *operand* di sisi kanan operator penugasan. Contoh penugasan telah kamu lakukan pada saat melakukan deklarasi variabel seperti:

$a = 5;$

Perhatikan dan ingat baik-baik bahwa makna dari simbol ‘=’ dalam pemrograman berbeda dengan tanda “sama dengan” dalam matematika yang menandakan persamaan!

2) Operator Aritmatika

Operator aritmatika digunakan untuk melakukan operasi matematika yang kamu kenal. Ada perbedaan notasi penulisan operator dengan yang kamu pelajari pada mata pelajaran Matematika yang dapat dilihat pada Tabel 2.9.



Tabel 2.9 Operator Matematika yang Umum Digunakan pada Bahasa

Operasi Matematika	Contoh Ekspresi Aljabar	Operator Bahasa Pemrograman	Ekspresi Bahasa Pemrograman
Penjumlahan	$1 + 2$	<code>+</code>	$1 + 2$
Pengurangan	$a - b$	<code>-</code>	$1 - b$
Perkalian	ab	<code>*</code>	$a * b$
Pembagian	a/b	<code>/</code>	a / b
Modulo (sisanya bagi)	$a \bmod b$	<code>%</code>	$a \% b$

Seperti pada matematika, suatu ekspresi dapat terdiri atas banyak operator. Agar ekspresi tersebut dieksekusi dengan benar, diperlukan suatu urutan operasi (atau *operator precedence*). Urutan operasi dari operator pada bahasa pemrograman seperti C ialah:

- Tanda kurung `()`
- Operator perkalian `*`, pembagian `/`, dan modulo `%`.
- Operator penjumlahan `+` dan pengurangan `-`

Selain operator yang kamu temukan di matematika, bahasa pemrograman juga memiliki operator lain seperti operator *left shift* `<<` dan *right shift* `>>` untuk mengalikan atau membagi nilai dengan faktor dua. Urutan operator yang lebih lengkap dapat dilihat pada dokumentasi bahasa pemrograman C yang dapat diakses di <https://buku.kemdikbud.go.id/s/xs8l5e>.

Hasil ekspresi dapat disimpan dalam suatu variabel menggunakan operator penugasan. Misalnya:

$a = 5;$

$b = a + 1;$

$b = b + 5;$

$c = (a + b) * 2 + a;$

$d = c \% b;$

Ekspresi seperti $b = b + 5$ mungkin tidak pernah kamu temui pada mata pelajaran Matematika. Akan tetapi, ekspresi ini sangat lazim ditemukan dalam menulis kode program. Makna dari ekspresi ini ialah kamu melakukan



perubahan pada nilai suatu variabel berdasarkan nilai variabel pada saat ini. Dalam hal ini, kamu mengubah nilai variabel b dengan menambahkan 5 terhadap variabel tersebut. Hal ini telah kamu lakukan pada saat melakukan penelusuran pada Diagram Alir 4 di bagian awal bab ini.

Untuk ekspresi seperti ini, bahasa C menyediakan bentuk yang lebih ringkas yang disebut sebagai *compound assignment operator* atau operator penugasan majemuk. Misalnya, $b = b + 5$ dapat disingkat menjadi $b += 5$. Hal ini juga berlaku untuk operator matematika lainnya seperti $b *= 2$, $c /= 3$, dan seterusnya.

3) Operator Increment dan Decrement

Selain operator penugasan majemuk pada bagian sebelumnya, terdapat operator unary yang lebih khusus untuk menambah (++) atau mengurangi nilai suatu variabel (--) dengan angka 1. Operator ini dapat diletakkan sebelum (*prefix*) atau setelah (*postfix*) *operand*. Pada penulisan dalam bentuk *prefix*, perubahan nilai akan langsung dilakukan pada nilai variabel sebelum nilai variabel tersebut digunakan pada ekspresi. Sebagai contoh, setelah dua baris kode berikut dijalankan, nilai x dan y akan bernilai 2.

```
x = 1;  
y = ++x;
```

Sebaliknya, pada penulisan dalam bentuk *postfix*, nilai variabel akan digunakan terlebih dahulu pada ekspresi, baru perubahan dilakukan.

Sebagai contoh, setelah dua baris kode berikut dijalankan, nilai x dan y akan bernilai 1.

```
x = 1;  
y = x++;
```

4) Operator Logika, Relasional, dan Kesamaan

Di samping operator aritmatika, juga dikenal operator logika, relasional, dan kesamaan. Ekspresi yang menggunakan operator ini akan memiliki nilai benar (*true* atau dalam bahasa C bernilai tidak sama dengan 0) atau salah (*false* atau bernilai 0). Operator pada kategori ini memiliki peran yang sama dengan operator logika, relasional, dan kesamaan pada mata pelajaran Matematika. Operator tersebut dapat dilihat pada Tabel 2.10.



Tabel 2.10 Operator Logika, Relasional, dan Kesamaan pada Bahasa Pemrograman

Aljabar	Program	Contoh	Makna
Operator Kesamaan/Pertidaksamaan			
=	==	$a == b$	Apakah nilai a sama dengan b ?
≠	!=	$a != b$	Apakah nilai a tidak sama dengan b ?
Operator Relasional			
>	>	$a > b$	Apakah a lebih besar daripada b ?
<	<	$a < b$	Apakah a lebih kecil daripada b ?
≥	>=	$a >= b$	Apakah a lebih besar atau sama dengan b ?
≤	<=	$a <= b$	Apakah a lebih kecil atau sama dengan b ?
Operator Logika			
AND	&&	$a > 0 \ \&\& \ b > 0$	Apakah a dan b lebih besar daripada 0?
OR		$a > 0 \ \ b > 0$	Apakah a atau b lebih besar daripada 0?
NOT	!	$!(a > 0)$	Apakah a tidak lebih besar daripada 0?

Praktik Baik Pemrograman

Pada saat menuliskan suatu ekspresi, gunakanlah pengelompokan dengan tanda kurung untuk membuat ekspresi tidak ambigu dan mudah untuk dibaca. Pastikan ekspresi tersebut ditulis dengan benar sebelum kamu menjalankan program. Biasakan untuk menguji (dalam pikiran kamu) sambil menulis, tidak menunggu program dijalankan.

Aktivitas 10.2-04-U Latihan Ekspresi

Buatlah solusi program untuk menyelesaikan problem-problem berikut. Kamu dapat menggunakan diagram alir, pseudokode, pemrograman visual, atau bahasa pemrograman untuk menghasilkan solusi dari problem tersebut sesuai dengan kondisi di tempat kamu.



Problem 1: Menghitung Luas Tanah (Tingkat Kesulitan: ★★)

Deskripsi Soal:

Pak Algor memiliki sebidang tanah berbentuk segitiga siku-siku. Dia ingin mengetahui berapa luas tanah yang dia miliki. Bantulah Pak Algor menghitung dengan membuat program untuk menghitung luas tanahnya.

Format Masukan:

Dua buah bilangan bulat yang merupakan alas dan tinggi segitiga dari tanah Pak Algor.

Format Keluaran:

Sebuah bilangan riil, dengan dua digit di belakang desimal yang merupakan luas tanah Pak Algor. Akhiri keluaran dengan karakter *newline*.

Masukan	Keluaran
20 40	400.00

Problem 2: Menghitung Luas Persegi (Tingkat Kesulitan: ★★)

Simak kembali Diagram Alir 1: Menghitung Luas Persegi pada bagian Algoritma di bab ini. Buatlah program berdasarkan diagram alir tersebut. Panjang sisi yang diberikan ialah sebuah bilangan riil dengan nilai maksimum 1.000 cm.

Problem 3: Hasil Bagi dan Sisa Pembagian (Tingkat Kesulitan: ★★★)

Deskripsi Soal:

Buatlah sebuah program untuk menampilkan hasil dan sisa pembagian dari dua buah bilangan bulat positif a dan b .

Format Masukan:

Dua buah bilangan positif a dan b yang dipisahkan oleh karakter spasi. Keduanya bernilai paling besar 10 miliar.

Format Keluaran:

Dua buah bilangan bulat yang ditulis di baris berbeda. Baris pertama ialah hasil pembagian, baris kedua ialah sisa pembagian. Hasil pembagian dibulatkan ke bawah.

Masukan	Keluaran
1000 3	333 1



Problem 4: Benar atau Salah? (Tingkat Kesulitan: ★ ★ ★)

Ekspresi yang memuat operator logika, relasional, dan kesamaan dapat kamu telusuri tanpa menjalankan program. Berikut ini diberikan beberapa ekspresi yang perlu kamu periksa nilainya, jika diketahui nilai $a = 1$, $b = 2$, dan $c = 3$.

No	Ekspresi	Hasil
1	$(a < b) \mid \mid (b > c)$	
2	$(a >= b) \mid \mid (b != c)$	
3	$(b == a) \&\& (c > a)$	
4	$(b >= a) \&\& ((b < c) \mid \mid (c > a))$	

3. Struktur Kontrol Percabangan

Pernahkah kamu ingin pergi ke sebuah tempat tertentu dengan menggunakan moda transportasi? Keputusan menggunakan sebuah moda transportasi untuk bepergian biasanya bergantung pada sebuah keadaan tertentu. Apabila kondisi hujan, kamu akan lebih memilih menggunakan mobil daripada menggunakan sepeda motor. Jika mau ke luar pulau, kamu lebih memilih menggunakan pesawat terbang. Apabila cuaca sedang cerah dan jarak yang ditempuh dekat, kamu akan memilih menggunakan sepeda motor.

Komputer merupakan alat yang membantu banyak aktivitas manusia. Saat ini, komputer merupakan bagian yang tidak terpisahkan dari kehidupan sehari-hari. Pada dasarnya, komputer menjalankan perintah dari manusia. Perintah-perintah tersebut dituangkan secara tertulis dalam sebuah aturan tertentu yang disebut kode program. Penyusunan sebuah kode program membutuhkan kemampuan algoritma, logika, dan bahasa pemrograman. Aktivitas membuat program merupakan latihan untuk menuliskan sebuah kode program sederhana dalam rangka memberikan instruksi ke komputer untuk melakukan sebuah aksi.

Kamu akan belajar memahami konsep pemrograman sederhana khususnya yang berkaitan dengan sebuah struktur kontrol percabangan atau kondisional. Apa itu kondisional? Secara sederhana, kondisional dalam dunia matematika dikenal dalam bentuk pernyataan "jika maka" (*if else statement*).



Pernyataan ini dibuat untuk mengekspresikan sebuah aksi berdasarkan sebuah kondisi tertentu. Sebagai contoh, ketika kamu diminta untuk mengklasifikasikan sebuah bilangan merupakan bilangan ganjil atau genap, dengan mudah kamu membuat sebuah aturan sebagai berikut.

- Jika bilangan tersebut habis dibagi 2, maka bilangan tersebut termasuk bilangan genap.
- Jika bilangan tersebut tidak habis dibagi 2, maka bilangan tersebut termasuk bilangan ganjil.

Proses tersebut merupakan salah satu ilustrasi dari sebuah pernyataan kondisi.

Dalam kehidupan sehari-hari, manusia perlu mengambil keputusan. Keputusan tersebut dapat berupa keputusan sederhana hingga keputusan yang sangat kompleks. Banyak permasalahan yang di dalamnya terkandung pengambilan keputusan sehingga keputusan pun juga digunakan dalam menyelesaikan suatu permasalahan komputasi. Pada bagian ini, kamu akan melihat bagaimana kamu dapat menerapkan suatu bentuk keputusan sederhana di dalam program. Pada bahasa pemrograman C, dan juga banyak bahasa pemrograman lainnya, keputusan dapat diimplementasikan dalam bentuk struktur kontrol percabangan. Ada beberapa variasi dari struktur ini, yaitu pernyataan *if-else*, pernyataan *switchcase*, dan pernyataan yang bersarang.

a. Struktur Kontrol Percabangan *If - Else*

Ada beberapa variasi penggunaan struktur kontrol percabangan *If - Else*. Bentuk umum dari pernyataan *if* ialah sebagai berikut.

```
if (kondisi) {  
    <pernyataan>;  
    <pernyataan>;  
    ...  
}
```

Bagian kondisi dapat diisi dengan ekspresi yang menghasilkan nilai benar atau salah. Apabila kondisi menghasilkan nilai benar, semua pernyataan yang berada di dalam struktur kontrol percabangan tersebut akan dieksekusi oleh program. Sekarang, perhatikan program berikut, dan lakukan penelusuran untuk memeriksa keluaran dari program tersebut.



```

/*
 * Program Membandingkan Bilangan
 */

#include <stdio.h>
int main(){
    int a = 1, b = 1;
    if (a == b) {
        printf("a sama dengan b \n");
    }
    return 0;
}

```

Program tersebut menggunakan struktur percabangan pada baris 5-7. Ekspresi yang digunakan pada bagian kondisi ialah $a==b$, sedangkan pernyataan yang dieksekusi jika kondisi benar terdapat pada baris 6. Karena hanya terdapat satu pernyataan yang menjadi bagian dari struktur kontrol tersebut, kamu dapat menghilangkan tanda kurung kurawal sehingga program dapat ditulis menjadi:

```

/*
 * Program Membandingkan Bilangan
 */

#include <stdio.h>

int main(){
    int a = 1, b = 1;
    if (a == b)
        printf("a sama dengan b \n");
    return 0;
}

```

Struktur kontrol percabangan dapat menambahkan blok *else* yang akan dieksekusi apabila kondisi bernilai salah.



```
if (kondisi)
    <pernyataan>;
else
    <pernyataan>;
```

Misalnya, untuk mengecek apakah suatu bilangan merupakan bilangan ganjil atau genap, kamu dapat memanfaatkan struktur *if-else* sebagai berikut:

```
/*
 * Program Cek Ganjil-Genap
 */

#include <stdio.h>

int main(){
    int bilangan;
    scanf("%d", &bilangan);
    if (bilangan % 2 == 0)
        printf("Bilangan Genap\n");
    else
        printf("Bilangan Ganjil\n");
    return 0;
}
```

Apabila kondisi makin kompleks, struktur *if-else* ini dapat dikembangkan kembali menjadi:

```
if (kondisi ke-1)
    <statement>
else if (kondisi ke-2)
    <statement>
.....
else if (kondisi ke-n )
    <statement>
else
    <statement>
```



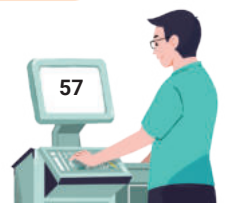
b. Struktur Kontrol Percabangan *Switch-Case*

Struktur kontrol percabangan yang memiliki cabang banyak dapat dibuat lebih sederhana menggunakan struktur *switch-case*. Bentuk umum dari struktur ini ialah sebagai berikut.

```
switch(switch_expr)
{
    case (constant expr1) :<statement>;
        <statement>;
        break;
    case (constant expr2) :<statement>;
        <statement>;
        break;
    ....
    default      : <statement>;
        <statement>;
        break;
}
```

Sebagai contoh, perhatikan kode program berikut.

```
#include <stdio.h>
int main()
{
    int bilangan, sisaPembagian;
    scanf("%d", &bilangan);
    sisaPembagian = bilangan % 4;
    switch (sisaPembagian)
    {
        case 0: printf("Habis Dibagi\n"); break;
        case 1: printf("Sisa Satu\n"); break;
        case 2: printf("Sisa Dua\n"); break;
        case 3: printf("Sisa Tiga\n"); break;
    }
    return 0;
}
```



Pada program di atas, struktur *switch-case* memeriksa nilai yang ada pada variabel sisa Pembagian. Karena nilai tersebut merupakan sisa pembagian sebuah bilangan dengan empat, hanya ada empat kemungkinan nilai, yaitu 0 sampai 3. Setiap kemungkinan nilai tersebut diperiksa melalui empat buah struktur *case* yang akan mencetak kalimat ke layar yang sesuai dengan sisa pembagian yang diperoleh.

c. Struktur Kontrol Percabangan Bersarang

Sebuah struktur kontrol dapat menjadi bagian dari suatu struktur kontrol lain. Hal ini disebut *nested* atau bersarang. Pada contoh berikut, diberikan sebuah kode program yang memiliki struktur kontrol percabangan bersarang. Telusurilah program tersebut jika program diberi masukan 1000 dan 10.

```
/*
 * Program dengan IF bersarang
 */

#include <stdio.h>
int main()
{
    int a, b;
    scanf("%d %d", &a, &b);
    if (b!=0)
        if (a/b>10)
            printf("1\n");
        else
            printf("-1\n");
    return 0;
}
```





Aktivitas 10.2-05-U

Latihan Struktur Percabangan

Buatlah solusi program untuk menyelesaikan problem-problem berikut.

Kamu dapat menggunakan diagram alir, pseudokode, pemrograman visual, atau bahasa pemrograman untuk menghasilkan solusi dari problem tersebut sesuai dengan kondisi di tempat kamu.

Problem 1: Membagi Bilangan (Tingkat Kesulitan: ★★)

Buatlah sebuah program dari Diagram Alir 3: Membagi Bilangan yang tersedia pada bagian Algoritma di awal bab ini.

Problem 2: Bilangan Bulat Positif (Tingkat Kesulitan: ★★)

Deskripsi Soal:

Buatlah program untuk mengecek apakah sebuah bilangan bulat ialah bilangan bulat positif.

- Jika bilangan bulat tersebut merupakan bilangan bulat positif, cetaklah “Bilangan Bulat Positif”.
- Jika bilangan bulat tersebut bukan merupakan bilangan bulat positif, jangan cetak apa pun.

Format Masukan:

Sebuah bilangan bulat n . Nilai n berada pada rentang $-100 < n < 100$.

Format Keluaran:

Satu baris kalimat sesuai pada deskripsi soal.

Masukan	Keluaran
3	Bilangan Bulat Positif
-10	

Problem 3: Jenis Bilangan Bulat (Tingkat Kesulitan: ★★★)

Deskripsi Soal:

Buatlah program untuk apakah sebuah bilangan bulat ialah bilangan termasuk bilangan bulat positif, negatif atau nol. Jika bilangan bulat tersebut merupakan bilangan bilangan bulat positif, cetak “Bilangan



Bulat Positif”. Jika bilangan bulat tersebut merupakan bilangan bulat negatif, cetak “Bilangan Bulat Negatif”. Jika bilangan bulat tersebut merupakan bilangan bulat nol, cetak “Bilangan Bulat Nol”.

Format Masukan:

Sebuah bilangan bulat n . Nilai n berada pada rentang $-100 < n < 100$.

Format Keluaran:

Satu baris kalimat sesuai pada deskripsi soal.

Masukan	Keluaran
3	Bilangan Bulat Positif
-5	Bilangan Bulat Negatif

Problem 4: Nama Bulan (Tingkat Kesulitan: ★★)

Deskripsi Soal:

Buatlah sebuah program yang menerima masukan bilangan bulat yang berada pada rentang 1–12, dan akan mencetak nama bulan yang sesuai dengan bilangan bulat tersebut. Apabila bilangan berada di luar rentang tersebut, cetak kalimat “Tidak ada bulan yang sesuai”.

Format Masukan:

Sebuah bilangan bulat n . Nilai n berada pada rentang $-100 < n < 100$.

Format Keluaran:

Satu baris kalimat sesuai pada deskripsi soal.

Masukan	Keluaran
1	Januari
3	Maret
13	Tidak ada bulan yang sesuai

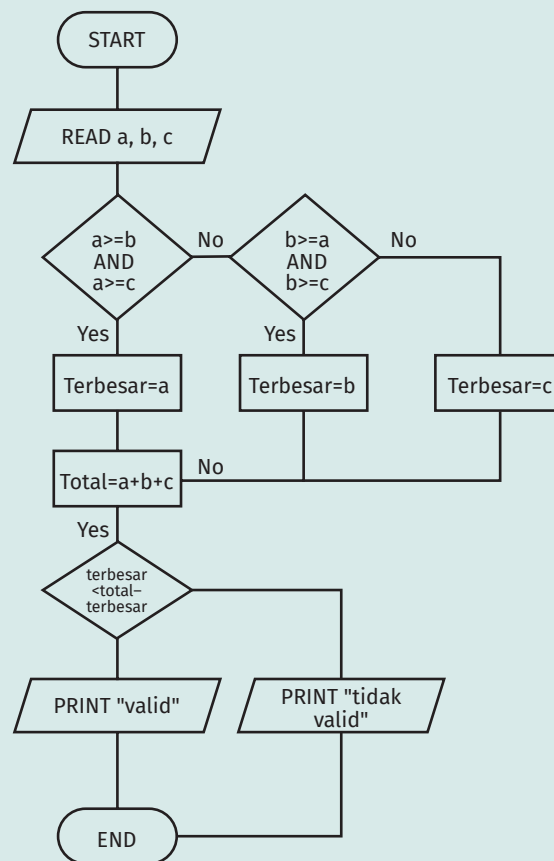
Problem 5: Mengecek Sisi Segitiga (Tingkat Kesulitan: ★★★)

Tahukah kamu bahwa sebuah segitiga hanya dapat dibangun apabila sisi terpanjangnya lebih kecil daripada total panjang kedua sisi lainnya? Jika syarat ini tidak dipenuhi, tidak ada segitiga yang terbentuk.



Agria sedang membuat program untuk menghitung luas segitiga yang menerima masukan berupa tiga buah bilangan bulat yang merupakan panjang sisi segitiga tersebut. Akan tetapi, Agria menyadari bahwa dia harus terlebih dahulu memastikan ketiga panjang sisi yang dimasukkan benar-benar dapat membentuk sebuah segitiga. Oleh karena itu, dia merancang sebuah algoritma dalam bentuk diagram alir berikut untuk mengecek apakah ketiga bilangan tersebut dapat membentuk segitiga.

Tugas kamu ialah membantu Agria dengan membuat program berdasarkan diagram alir tersebut.



Problem 6: Belajar Membuat Kasus Uji (Tingkat Kesulitan: ★★)

Perhatikan kembali diagram alir pada Problem 5. Diagram alir tersebut terlihat memiliki beberapa kemungkinan aliran, bergantung pada nilai masukan yang diberikan. Saat kamu mengecek program



yang kamu buat dengan suatu kasus uji, kasus uji yang diberikan haruslah meliputi semua kemungkinan aliran tersebut. Sekarang, buatlah kasus uji sedemikian rupa sehingga semua kemungkinan aliran pada diagram alir di atas dapat dicek.

Ayo, Renungkan!

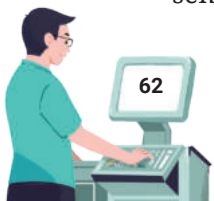
Jawablah pertanyaan berikut dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal.

1. Pada bagian ini, kamu mendapatkan banyak konsep baru tentang program. Seperti apa perasaanmu saat ini?
2. Apakah kamu bereksperimen dengan contoh-contoh yang diberikan di buku? Jika ya, pengetahuan paling menarik apa yang kamu temukan dari hasil eksperimen tersebut?
3. Dari latihan yang telah kamu kerjakan, terutama dari problem 1 dan 5, apa salah satu contoh penggunaan dari struktur kontrol percabangan?
4. Apakah kamu sudah memahami proses membuat kasus uji untuk menguji sebuah program, yang harus meliputi semua kemungkinan aliran pada diagram alir? Mengapa kamu harus memastikan semua kemungkinan aliran dicek?
5. Kesalahan apa yang sering kamu lakukan saat menulis kode program dengan menggunakan struktur kontrol percabangan?

4. Struktur Kontrol Perulangan

Kamu akan belajar memahami konsep dan cara kerja perulangan dalam pemrograman. Setelah menyelesaikan aktivitas ini, kamu diharapkan mampu menerapkan perulangan dan menghubungkannya dengan kejadian sehari-hari.

- Pernahkah kamu melakukan aktivitas yang sama berulang kali? Eits... tidak perlu jauh-jauh, untuk melangkah saja, kita mengulang pergerakan kaki kanan dan kiri. Apakah kamu selalu berpikir sebelum melakukan setiap langkah? Pertanyaan terakhir tersebut dapat ditanyakan pada semua kegiatan perulangan yang kamu lakukan.



- Membaca buku, — kamu membalikkan halaman buku setiap selesai membaca halaman tersebut. Berangkat ke sekolah, — kamu melalui jalur yang terencana setiap hari. Bermain *game*, — kamu melakukan hal yang sama berulang-ulang.
- Tidak hanya itu, perulangan juga merupakan inti dalam berhitung. Jika kamu menghitung 5×3 , apa saja langkah yang kamu lakukan untuk mendapatkan hasilnya?

Salah satu keunggulan program komputer daripada manusia ialah kemampuannya untuk mengolah data yang berukuran besar atau melaksanakan suatu aksi berulang kali dalam periode waktu yang lama tanpa merasa bosan atau lelah. Hal ini dimungkinkan dengan adanya suatu kontrol perulangan. Pernyataan perulangan atau *loop* merupakan struktur program untuk keperluan iterasi, yaitu memproses satu atau beberapa pernyataan secara berulang (*looping*) berdasarkan kondisi tertentu. Program C menyediakan tiga bentuk pernyataan *loop*, yaitu:

- *for loop*
- *while loop*
- *do...while loop*

a. Struktur Kontrol Perulangan *For-Loop*

Pernyataan ini umumnya digunakan untuk memproses pernyataan secara berulang-ulang, dengan jumlah perulangan yang dilakukan telah diketahui sebelumnya. Misalnya, berjalan sebanyak n langkah ke depan, atau mencetak barisan dari suku pertama hingga suku ke- n . Struktur kontrol perulangan *for* ialah sebagai berikut.

```
for (expr1; expr2; expr3)
{
    <statement>;
    ...
}
```

Struktur di atas akan dijalankan melalui proses berikut.

- Ekspresi *expr1* akan dieksekusi ketika program menjalankan struktur *for* tersebut. Ekspresi ini biasanya berisi inisialisasi suatu variabel *counter* yang digunakan untuk menghitung jumlah perulangan yang telah dilakukan.



- Ekspresi `expr2` ialah suatu ekspresi bernilai benar atau salah (*boolean*) yang akan dicek sebelum pernyataan di dalam blok struktur dieksekusi. Apabila ekspresi ini bernilai benar, pernyataan akan dieksekusi. Sebaliknya, apabila ekspresi bernilai salah, pernyataan tidak akan dieksekusi dan perulangan berakhir. Dengan kata lain, pada bagian ini, kamu menuliskan sebuah pernyataan yang merupakan kondisi berhenti (*stopping criteria*) untuk memastikan perulangan yang kamu buat memiliki langkah yang terbatas (dipastikan berhenti).
- Ekspresi `expr3` merupakan sebuah pernyataan yang dijalankan setelah semua pernyataan di dalam struktur *for* dieksekusi. Biasanya, pernyataan ini dibuat untuk mengubah nilai variabel *counter* yang akan makin mendekati kondisi berhenti (memastikan nilai *counter* akan konvergen ke kondisi berhenti).

Walaupun ketiga ekspresi tersebut bersifat opsional (tidak harus ada), kamu disarankan untuk menuliskan ketiga ekspresi tersebut saat membuat program dengan jelas dan lengkap. Untuk saat ini, kamu perlu memahami teknik untuk menulis ketiga ekspresi tersebut dengan baik.

Perhatikan contoh kode program menulis bilangan bulat berikut. Pada contoh tersebut, program akan menulis bilangan bulat dari 0 hingga kurang dari *n*. Telusurilah kode program tersebut dengan teknik penelusuran yang telah kamu pelajari pada saat menelusuri suatu diagram alir!

```
/*
 * Program Menulis Bilangan Bulat sebanyak n kali
 */

#include <stdio.h>
int main()
{
    int i, n;

    scanf("%d", &n);
    for (i=0; i < n; i=i+1) // i = i+1 dapat juga ditulis i++
    {
        printf("%d\n", i);
    }
    return 0;
}
```



Keluaran program tersebut saat diberi masukan 5 ialah:

0
1
2
3
4

Pada contoh tersebut, variabel *counter* yang digunakan ialah *i* yang nilainya diinisialisasi (diberi nilai awal) dengan nilai 0. Pada bahasa C, lazimnya *counter* dimulai dari nilai 0, berbeda dengan proses pencacahan yang biasa kamu lakukan dari 1 di dunia nyata. Hal ini terkait dengan beberapa aspek teknis di bahasa C yang dimulai dari 0. Misalnya, di kelas XI nanti, kamu akan menggunakan struktur data *array* yang dimulai dari indeks ke-0. Perlu diingat bahwa inisiasi dengan 0 ini hanyalah suatu kebiasaan masyarakat pemrogram dalam bahasa C, dan kamu tetap dapat melakukan pencacahan mulai dari 1. Kamu dapat mencoba mengubah kode program di atas sehingga *counter* berjalan dari 1. Selain *counter* yang berjalan menaik (*ascending*), kamu juga dapat membuat suatu *counter* yang berjalan menurun (*descending*). Contoh ini disajikan misalnya pada diagram alir keempat dan kelima pada bagian Algoritma.

Pernyataan yang ada pada *expr1* hingga *expr3* dapat ditulis menjadi deretan instruksi yang dipisahkan dengan tanda koma. Misalnya, terdapat pada contoh program berikut.

```
1  for (i=0, j=0; i<3; ++i, j++)  
2  {  
3      printf("%d %d", i, j);  
4  }
```

Urutan pengerjaan akan sama seperti pada contoh sebelumnya. Akan tetapi, ada dua pernyataan yang akan dijalankan pada *expr1* hingga *expr3*.

Praktik Baik Pemrograman

Pada saat merancang sebuah struktur perulangan *for*, kamu perlu memastikan agar kondisi berhenti pasti akan tercapai (konvergen). Apabila kondisi berhenti tidak pernah tercapai, akibatnya, struktur ini akan berjalan terus-menerus



dan menyebabkan terjadinya suatu perulangan yang tidak terbatas (*infinite loop*). Apabila hal ini terjadi, program akan dibekukan oleh sistem operasi, bahkan akan dihentikan. Perhatikan contoh berikut.

```
/*  
 * Program dengan Perulangan tak Terbatas  
 */  
  
#include <stdio.h>  
  
int main()  
{  
    int i, n;  
    scanf("%d", &n);  
    for(i=0; i<n; i--)  
        printf("%d\n", n)  
    return 0;  
}
```

Dapat dilihat bahwa pada kode tersebut, nilai *counter* akan berkurang dan tidak akan pernah melebihi nilai *n* jika *n* diisi dengan suatu bilangan bulat positif.

Ayo, Cari Tahu!

Lakukan penelusuran pada kode program tersebut dengan nilai $n = 3$.

b. Struktur Kontrol Perulangan *While*

Saat merancang perulangan, kamu bisa jadi tidak dapat menentukan berapa kali perulangan akan dilakukan. Akan tetapi, kamu mengetahui kondisi berhentinya. Misalkan, instruksi berikut pada dunia nyata, “berjalan luruslah sampai ujung jalan, kemudian belok kiri”. Instruksi tersebut tidak memberikan gambaran jelas jumlah langkah yang akan kamu lakukan. Namun, secara naluriah, kamu mengetahui kapan kamu harus berhenti berjalan lurus, lalu berbelok ke arah kiri.



Pada program, suatu struktur kontrol *while* dikenal untuk melakukan perulangan seperti pada contoh di atas. Struktur kontrol tersebut dapat ditulis sebagai berikut. Pernyataan akan dieksekusi terus-menerus selama ekspresi kondisi bernilai benar.

```
while (ekspresi kondisi) {  
    <pernyataan>;  
    ...  
}
```

Sebagai contoh, misalnya kamu akan menulis kode program untuk membaca dan menuliskan kembali bilangan bulat positif. Hal ini terus dilakukan hingga program membaca nilai -1. Program tersebut dapat kamu lihat di bawah ini.

```
/*  
 * Program Baca Tulis Bilangan  
 */  
#include <stdio.h>  
int main()  
{  
    int bilangan;  
    scanf("%d", &bilangan);  
    while (bilangan != -1) {  
        printf("%d\n", bilangan);  
        scanf("%d", &bilangan);  
    }  
    return 0;  
}
```

Ayo, Cari Tahu!

Lakukan penelusuran pada kode program ini dengan masukan yang diberikan ialah 1 2 3 4 -1



c. Struktur Kontrol Perulangan *Do – While*

Struktur kontrol *do-while* memiliki perilaku yang mirip dengan *while*, yaitu kamu hanya mengetahui kondisi berhenti dari perulangan tersebut. Perbedaannya ialah struktur *do-while* dipastikan akan dikerjakan setidaknya satu kali. Bentuk umum pernyataan *do .. while* ialah sebagai berikut.

```
do {  
    <pernyataan>;  
} while (ekspresi kondisi);
```

Salah satu contoh penggunaan struktur *do-while* ialah ketika kamu menulis sebuah program interaktif yang akan meminta pengguna memasukkan kembali suatu nilai hingga nilai tersebut memenuhi suatu syarat. Hal ini akan sering kamu alami ketika kamu diminta untuk mengisi ulang (*retry*) saat menggunakan sebuah program atau mengisi sebuah formulir elektronik.

Misal, program berikut akan terus meminta pengguna memasukkan nilai sampai pengguna tersebut memasukkan bilangan bulat positif.

```
/*  
 * Program Verifikasi Masukan Pengguna  
 */  
  
#include <stdio.h>  
  
int main()  
{  
    int bilangan;  
    do  
    scanf("%d\n", &bilangan);  
    while (!(bilangan > 0));  
  
    printf("Anda telah memasukkan bilangan bulat positif\n");  
    return 0;  
}
```



d. Struktur Kontrol Perulangan Bersarang

Sama seperti struktur kontrol percabangan, kamu dapat meletakkan struktur kontrol perulangan secara bersarang. Misalnya, pada contoh program berikut yang akan mencetak suatu pola berbentuk persegi menggunakan karakter asterisk “*”.

```
/*
 * Program dengan FOR Bersarang
 */

#include <stdio.h>
const int PANJANG = 2;
const int TINGGI = 3;

int main()
{
    int i, j;
    for (i = 0; i < TINGGI; i++) {
        for (j = 0; j < PANJANG; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Ayo, Cari Tahu!

Lakukanlah penelusuran pada kode program tersebut!

Pada contoh di atas, *counter* pada struktur *for* terluar dan terdalam tidak saling berkaitan. Pada beberapa kasus nantinya, kedua *counter* tersebut bisa saja saling berkaitan. Misalnya, pada program berikut.

```
/*
 * Program dengan Dua Counter
 */
```



```
#include <stdio.h>
int main()
{
    int i, j, m, n;
    scanf("%d %d", &m, &n);
    for (i=1; i<=m; i++) {
        for (j=i; j<=n; j++) {
            printf("%d", j);
            if (j==n)
                printf("\n");
            else
                printf(" ");
        }
    }
    return 0;
}
```

Pada program tersebut, kamu dapat melihat bahwa nilai awal *counter* j akan berpengaruh pada nilai *counter* i. Saat dijalankan, keluaran dari program tersebut ialah:

```
1 2 3 4 5
2 3 4 5
3 4 5
```

Tentunya, tidak ada batasan jumlah struktur yang kamu buat secara bersarang. Kamu pun juga dapat memadukan struktur perulangan dengan struktur percabangan sehingga menghasilkan program yang lebih kompleks.



Aktivitas 10.2-06-U Latihan Struktur Perulangan

Buatlah solusi program untuk menyelesaikan problem-problem berikut. Kamu dapat menggunakan diagram alir, pseudokode, pemrograman visual, atau bahasa pemrograman untuk menghasilkan solusi dari problem tersebut sesuai dengan kondisi di tempatmu.



Problem 1: Menghitung Mundur (Tingkat Kesulitan: ★★)

Buatlah kode program berdasarkan Diagram Alir 4 pada bagian Algoritma untuk mencetak bilangan secara hitung mundur.

Problem 2: Menghitung Rataan (Tingkat Kesulitan: ★★★)**Deskripsi Soal:**

Buatlah sebuah program yang akan menghitung rata-rata dari n buah bilangan.

Format Masukan:

Baris pertama ialah sebuah bilangan bulat positif n yang menunjukkan banyaknya data, baris berikutnya berisi n buah bilangan bulat. Nilai n maksimal 1000, dan besarnya bilangan yang harus dihitung rata-ratanya berada pada rentang -1 miliar hingga 1 miliar.

Format Keluaran:

Nilai rata-rata dari n buah bilangan masukan. Nilai rata-rata tersebut dituliskan sebagai bilangan riil dengan dua angka di belakang titik desimal.

Masukan	Keluaran
10 20 30	20.00

Problem 3: Mencari Bilangan Terbesar (Tingkat Kesulitan: ★★★)

Buatlah kode program berdasarkan Diagram Alir 5 pada bagian Algoritma untuk mencari bilangan terbesar dari sekumpulan bilangan yang diberikan.

Problem 4: Membuat Mesin Sortir Kembang Kol (Tingkat Kesulitan: ★★★★★)**Deskripsi Soal:**

Kamu akan membantu seorang petani kembang kol untuk menyortir kembang kol yang telah dipanen berdasarkan ukurannya. Kembang kol tersebut akan dikelompokkan menjadi berukuran kecil (< 50 gram per buah), berukuran sedang (50-200 gram per buah), dan berukuran jumbo (> 200 gram per buah). Selama ini, petani tersebut menyortir kembang kol menggunakan tenaga manusia. Karena kamu telah memiliki kemampuan untuk membuat program yang dapat melakukan hal tersebut secara otomatis, kamu pun mengajak petani tersebut untuk menerapkan *future practice* dengan menerapkan otomatisasi. Bantulah



petani tersebut dengan program buatan kamu! Ini akan menjadi langkah awal bagi petani tersebut untuk menjadi seorang petani modern.

Format Masukan:

Program kamu akan membaca berat dari setiap kembang kol dalam satuan gram. Berat setiap kembang kol ditulis dalam bilangan riil. Apabila semua kembang kol telah tersortir, program kamu akan membaca nilai -1.

Format Keluaran:

Tiga buah bilangan bulat yang merupakan jumlah kembang kol pada setiap kategori, yaitu kecil, sedang, dan jumbo. Setiap bilangan ditulis pada baris yang berbeda.

Masukan	Keluaran
100.0 20.5 300.1 40.1 -1	2 1 1

Problem 5: Menggambar Segitiga (Tingkat Kesulitan: ★★★★★)

Deskripsi Soal:

Buatlah sebuah program untuk menggambar sebuah segitiga dengan menggunakan karakter asterisk (*).

Format Masukan:

Sebuah bilangan bulat n yang menyatakan ukuran segitiga yang akan dibuat. Nilai n berada pada rentang 1 hingga 20.

Format Keluaran:

Bentuk segitiga berukuran $n \times n$ dengan menggunakan karakter asterisk (*) seperti pada contoh keluaran.

Masukan	Keluaran
7	<pre> ***** ***** ***** **** *** ** * </pre>

Problem 6: Memperbaiki Program (Tingkat Kesulitan: ★★★)



Deskripsi Soal:

Buatlah sebuah program untuk menggambar sebuah pola X dengan menggunakan karakter asterisk (*).

Format Masukan:

Sebuah bilangan bulat n yang menyatakan ukuran pola yang akan dibuat. Nilai n berada pada rentang 1 hingga 20.

Format Keluaran:

Pola huruf X yang sesuai dengan ukuran yang dimasukkan. Perhatikan perbedaan untuk banyak baris ganjil dan genap pada contoh keluaran.

Masukan	Keluaran
5	*__* _**_ __*_ _**_ *__*
6	*___* _*__* __**_ _*__* *___*

Pada soal ini, kamu telah diberikan salah satu program yang dibuat untuk menyelesaikan permasalahan di atas. Akan tetapi, program tersebut tidak dapat dijalankan dan ditulis dengan tidak rapi! Pelajarilah kode program di bawah ini, dan perbaiki program tersebut hingga dapat berjalan dengan benar. Sesuaikan pula penulisannya dengan mengikuti praktik baik yang telah kamu pelajari hingga saat ini.

5. Fungsi

Selama ini, kamu telah sering bertemu dengan fungsi. Di matematika, kamu membuat suatu persamaan menggunakan fungsi seperti $y = f(x) = x + 1$. Fungsi $f(x)$ menerima sebuah masukan x yang disebut sebagai fungsi daerah dan menghasilkan nilai y yang merupakan fungsi wilayah. Fungsi sangat berhubungan dengan kemampuan abstraksi yang telah kamu pelajari di Berpikir Komputasional sehingga program yang kamu tulis dapat ditulis dengan lebih baik.



Sejauh ini, kamu juga telah menggunakan beberapa fungsi dalam berlatih pemrograman. Pada diagram alir, kamu telah mengenal sebuah simbol subprogram untuk memberikan abstraksi dari suatu proses lain yang kamu gunakan dalam solusimu. Pada program bahasa C, struktur *main* merupakan sebuah fungsi yang akan dieksekusi oleh sistem operasi ketika program dijalankan. Selain itu, kamu pun telah menggunakan fungsi seperti *printf* dan *scanf*.

Pada hakikatnya, fungsi pada program melambangkan suatu kumpulan pernyataan yang memiliki tujuan tertentu. Tujuan tersebut direpresentasikan oleh nama dari fungsi tersebut. Misalnya, *scanf* yang memiliki fungsi untuk membaca (*scan*) nilai dari pengguna. Fungsi juga dapat menerima parameter-parameter, dan juga dapat mengembalikan suatu nilai. Dengan membungkus kumpulan instruksi tadi ke dalam suatu fungsi, kamu dapat menggunakan kembali fungsi tersebut di berbagai lokasi dalam programmu.

a. Membuat Fungsi

Saat membuat suatu fungsi baru, kamu perlu menentukan tiga hal berikut: nama fungsi yang merepresentasikan tujuan dari fungsi, parameter yang dimasukkan ke dalam fungsi, serta nilai yang dikembalikan. Ketiga informasi ini disebut prototipe dari fungsi. Adapun pernyataan-pernyataan yang ada di dalam fungsi tersebut disebut implementasi dari fungsi.

Misal, kamu akan membuat sebuah fungsi untuk menghitung luas lingkaran. Maka, kamu perlu menetapkan ketiga hal tersebut dan menghasilkan prototipe fungsi berikut. Dari prototipe tersebut, kamu dapat melihat bahwa fungsi *hitungLuasLingkaran* memerlukan sebuah parameter dengan tipe data *float* yang merupakan radius dari lingkaran. Saat dipanggil, fungsi ini akan mengembalikan sebuah nilai *float* yang merupakan luas lingkaran.

```
float hitungLuasLingkaran(float radius);
```

Jika dituliskan lengkap dengan implementasinya, fungsi tersebut dapat ditulis menjadi:



```
float hitungLuasLingkaran(float radius){
    float luas;
    luas = 3.14 * radius * radius;
    return luas;
}
```

Pada kode program tersebut, kata kunci *return* digunakan untuk mengakhiri fungsi dan mengembalikan suatu nilai. Dalam hal ini, nilai yang dikembalikan ialah luas lingkaran.

Perlu diingat bahwa prototipe dari fungsi-fungsi berikut akan dianggap sebagai fungsi yang berbeda karena memiliki parameter yang berbeda dari tipe data. Hal ini disebut sebagai *overloading*.

```
float hitungLuasLingkaran(float radius);
float hitungLuasLingkaran(int radius);
```

Kamu juga dapat membuat fungsi dengan jumlah parameter lebih dari satu, misalnya untuk menghitung luas persegi panjang berikut:

```
float hitungLuasPersegiPanjang(float panjang, float lebar);
```

Tentunya, kamu juga dapat membuat fungsi yang tidak memiliki parameter masukan, seperti yang kamu lakukan pada saat membuat fungsi **main()** pada program.

b. Memanggil Fungsi

Setelah dibuat, fungsi dapat dipanggil (*function call*) di dalam kode program. Perhatikan kode program berikut, yang akan memanggil fungsi *hitungLuasLingkaran* yang telah dibuat.

```
/*
 * Program Hitung Luas Lingkaran
 */

#include <stdio.h>

float hitungLuasLingkaran(float radius){
    float luas;
```



```

    luas = 3.14 * radius * radius;
    return luas;
}

int main(){
    float radius, luas;
    scanf("%f", &radius);
    luas = hitungLuasLingkaran(radius);
    printf("%f\n", luas);
    return 0;
}

```

Kamu juga dapat memanggil suatu fungsi di fungsi lain yang kamu buat. Misalnya, kamu ingin membuat fungsi untuk menghitung luas permukaan bola. Kamu dapat menulisnya menjadi:

```

/*
 * Program Hitung Luas Bola
 */

#include <stdio.h>

float hitungLuasLingkaran(float radius){
    float luas;
    luas = 3.14 * radius * radius;
    return luas;
}

float hitungLuasBola(float radius){
    float luas;
    volume = 4.0 * hitungLuasLingkaran(radius);
    return luas;
}

int main(){
    float radius, luasBola;
    scanf("%f", &radius);
    luasBola = hitungLuasBola(radius);
    printf("%f\n", luasBola);
}

```



```
    return 0;
}
```

c. Variabel Lokal pada Fungsi

Pada contoh-contoh di atas, kamu akan menemukan adanya deklarasi variabel dengan *identifier* yang sama. Misalnya, variabel luas kamu temukan di dalam fungsi *hitungLuasLingkaran* dan *hitungLuasBola*. Kedua variabel tersebut disimpan pada alamat memori yang berbeda yang hanya dapat diakses di dalam fungsi tempat variabel tersebut berada. Dengan kata lain, keduanya ialah variabel lokal yang tidak saling berhubungan.

d. Fungsi Rekursif

Pada persamaan di mata pelajaran Matematika, kamu mungkin pernah menemukan suatu fungsi yang ditulis secara rekursif seperti berikut.

$$f(n) = \begin{cases} 3 & , n = 0 \\ 2f(n-1) & , n < 0 \end{cases}$$

Pada contoh tersebut, terlihat bahwa $f(n)$ akan bernilai 3 jika nilai $n = 0$. Jika tidak, $f(n)$ akan memanggil fungsi $f(n-1)$ di dalamnya. Suatu fungsi yang memanggil dirinya sendiri disebut sebagai *fungsi rekursif*. Suatu fungsi rekursif harus memiliki dua buah bagian, yang disebut langkah basis dan langkah rekursif.

Langkah basis adalah bagian fungsi yang tidak memanggil dirinya sendiri. Pada contoh di atas, langkah basis dilakukan ketika n bernilai 0. Saat langkah basis ini dipanggil, rekursi akan selesai. *Langkah rekursif* adalah bagian fungsi yang memanggil dirinya sendiri. Pada contoh di atas, langkah rekursif dilakukan ketika n bernilai > 0 .

Pada saat merancang sebuah fungsi rekursif, kamu perlu memastikan agar langkah rekursif *harus* mencapai langkah basis. Pada contoh di atas, apabila nilai $n > 0$, langkah rekursif dipastikan akan mencapai langkah basis. Perhatikan bahwa parameter n selalu berkurang 1. Akan tetapi, jika n bernilai negatif, langkah rekursif tidak akan mencapai langkah basis. Oleh karena itu, penting bagi kamu untuk menspesifikasikan dengan jelas rentang parameter fungsi rekursif, langkah rekursif, dan langkah basis yang digunakan.



Fungsi rekursif di atas dapat ditulis seperti berikut dalam bentuk kode program.

```
/*  
 * Program dengan Fungsi Rekursif  
 */  
  
#include <stdio.h>  
int f(int n){  
    if (n == 0)           // Langkah Basis  
        return 3;  
    else  
        return 2 * f(n-1) + 4; // Langkah Rekursif  
}  
  
int main(){  
    printf("%f\n", f(5));  
    return 0;  
}
```

Praktik Baik Pemrograman

Gunakan fungsi untuk melakukan abstraksi. Kumpulkan fungsi-fungsi yang telah kamu buat agar dapat digunakan kembali untuk membuat program dengan lebih cepat. Kumpulan fungsi ini dapat kamu satukan menjadi sebuah pustaka atau *library*. Apabila pustaka tersebut memiliki manfaat yang besar dan dibutuhkan oleh banyak orang, kamu dapat membuat pustaka tersebut menjadi publik.



Aktivitas 10.2-07-UP Latihan Fungsi

Buatlah solusi program untuk menyelesaikan problem-problem berikut. Kamu dapat menggunakan diagram alir, pseudokode, pemrograman visual, atau bahasa pemrograman untuk menghasilkan solusi dari problem tersebut sesuai dengan kondisi di tempatmu.

1. Buatlah kode program dari Diagram Alir 2: Menghitung Luas Permukaan Kubus.



2. Buatlah sebuah fungsi untuk menghitung luas dan keliling bangun datar, seperti persegi panjang, lingkaran, atau segitiga.
3. Buatlah sebuah fungsi untuk menghitung luas permukaan bangun ruang seperti balok, kerucut, bola, dan limas.
4. Buatlah sebuah fungsi rekursif untuk menghitung faktorial.



Aktivitas 10.2-08-UP

Membuat Kasus Uji

Misalkan, kamu akan membuat suatu algoritma untuk menghitung akar dari persamaan kuadrat dengan menggunakan rumus berikut:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Jawablah pertanyaan-pertanyaan berikut.

1. Apakah yang menjadi masukan, proses, dan keluaran dari algoritma tersebut?
2. Apa saja kondisi yang dapat menyebabkan rumus di atas tidak dapat dikerjakan?

Berdasarkan jawaban kamu, lengkapilah tabel rancangan kasus uji berikut.

Masukan			Luaran / Perilaku Program
a	b	c	
1	3	-4	-4 dan 1
...	...	...	...
...	...	...	...
...	...	...	Tidak dapat dihitung karena melibatkan akar bilangan negatif.
...	...	...	Tidak dapat dihitung karena melibatkan pembagian dengan nol.

Dari hasil di atas, menurut kamu, apakah diperlukan adanya pengecekan terhadap nilai-nilai yang dimasukkan pada formula untuk memastikan



algoritma yang kamu rancang dapat berjalan dan menghasilkan keluaran yang benar?

C. Memilih Algoritma untuk Masalah di Kehidupan Nyata

Dalam kehidupan sehari-hari, kamu akan menemui berbagai permasalahan yang perlu kamu selesaikan. Untuk mencari solusi yang terbaik dari permasalahan tersebut, kamu perlu memilih dari berbagai pilihan langkah yang dapat digunakan. Begitu juga dengan algoritma yang digunakan dalam menyelesaikan permasalahan komputasional. Dalam menyelesaikan permasalahan tersebut, kamu perlu mencari dan memilih algoritma yang paling tepat dan sesuai untuk menyelesaikannya.

1. Pencarian (*Searching*)

Hidup ialah pencarian yang tiada henti. Mari, kita berpikir ke pengalaman “mencari” dalam kehidupan sehari-hari, misalnya seperti berikut ini.

- Pernahkah kamu merasa kebingungan saat mencari sebuah buku di lemari bukumu? Atau, bahkan, di perpustakaan sekolahmu? Saat kamu meminta bantuan kepada petugas, mengapa dia dapat menemukan buku yang kamu cari dengan waktu yang lebih singkat?
- Suatu hari, kamu kehilangan baju yang kamu butuhkan dan kamu mencarinya. Apa strategimu supaya baju tersebut cepat ditemukan?
- Kamu mengingat sebuah potongan lirik lagu, tetapi tidak ingat judul lagu tersebut. Bagaimana kamu dapat menemukan lagu tersebut dengan cepat?

Apa itu *mencari*? Mencari adalah usaha menemukan “sesuatu” yang dapat berupa benda, angka, konsep, informasi yang memenuhi kriteria tertentu dalam suatu ruang pencarian. Masalah pencarian sangat umum ditemukan di dalam kehidupan, termasuk dalam dunia komputasi. Ketika melakukan suatu pencarian, kamu harus menemukan suatu benda atau objek yang memenuhi kriteria tertentu dari sekumpulan benda atau objek lain. Beberapa contoh dari masalah pencarian yang sering kamu temui ialah sebagai berikut.

- Mencari buku dengan judul tertentu di rak buku perpustakaan.
- Mencari pakaian batik seragammu di lemari yang berisi semua pakaian yang kamu miliki.
- Mencari dokumen atau web tertentu dengan mesin pencari seperti Google.



Mencari benda nyata gampang, tinggal kamu lihat dan cocokkan dengan mata. Namun, mencari informasi atau konsep yang tidak kelihatan? Hmmmm... Tidak mudah!



(a)



(b)

Gambar 2.2 Pencarian (a) buku di perpustakaan, (b) informasi di internet.

Sumber: (a) Dokumen Kemdikbudristek, (b) Anthony Shkraba/Pexels.com

Masalah pencarian dapat dibuat dalam bentuk yang lebih formal agar dapat diterapkan pada banyak kasus. Elemen pada masalah pencarian seperti berikut.

1. Sekumpulan benda atau objek.
2. Kriteria dari benda atau objek yang dicari.
3. Pengecekan benda atau objek, untuk memeriksa apakah ia memenuhi kriteria pencarian.

Pertanyaan selanjutnya ialah bagaimana *strategi* untuk mencari. Tentunya, akan ada banyak cara yang dapat kita lakukan. Misalnya, kamu dapat mengambil pakaian secara acak dan mengecek apakah pakaian tersebut ialah seragam batik. Jika tidak, kamu akan mengembalikan pakaian tersebut kembali ke lemari dan kembali mengambil sebuah pakaian secara acak. Tentu ada cara lain, misalnya dengan memeriksa pakaian dari yang berada paling atas ke paling bawah. Atau, jika pakaian tersimpan dalam keadaan tergantung, kamu dapat mengecek dari pakaian paling kiri ke paling kanan.

Tentunya, ada banyak strategi lain yang dapat kamu gunakan. Ada strategi yang lebih baik daripada strategi yang lain bergantung pada keadaan benda atau objek tersebut saat pencarian dilakukan. Tentunya, kamu akan lebih mudah mencari suatu buku dengan judul tertentu di lemari perpustakaan yang tersusun



rapi dengan aturan tertentu dibandingkan dengan mencarinya di sebuah lemari yang berantakan.

2. Pengurutan (*Sorting*)

Saat merapikan sesuatu, misalnya koleksi buku, kamu menyusun buku tersebut dengan menggunakan suatu aturan. Misalnya, jika kamu memiliki koleksi buku cerita berseri, kemungkinan besar, kamu akan menyusunnya secara berurut dari volume pertama hingga volume yang terbaru. Atau, ketika sedang berbaris, kamu diminta untuk membentuk barisan berdasarkan tinggi badan. Hal-hal tersebut merupakan sebuah proses pengurutan atau *sorting*. Proses pengurutan akan menjadi bagian yang tidak terpisahkan dari program komputer atau aplikasi yang sering digunakan. Pada aktivitas ini, kamu akan melihat bagaimana proses pengurutan dapat dilakukan dengan menggunakan berbagai strategi. Pelajarilah strateginya!

Pengurutan ialah suatu permasalahan klasik pada komputasi yang dilakukan untuk mengatur agar suatu kelompok benda, objek, atau entitas diletakkan mengikuti aturan tertentu. Urutan yang paling sederhana misalnya mengurutkan angka secara terurut menaik atau menurun.

Biasanya, masalah pengurutan terdiri atas sekumpulan objek yang disusun secara acak yang harus diurutkan. Setelah itu, secara sistematis, posisi objek diperbaiki dengan melakukan pertukaran posisi dua buah objek. Hal ini dilakukan secara terus-menerus hingga semua posisi objek benar.

Misal, kamu memperoleh 5 buah angka acak berikut:



Kamu dapat membuat angka tersebut terurut menaik dengan melakukan satu kali pertukaran, yaitu dengan menukar nilai 4 dengan nilai 3. Terdapat 2 langkah penting dalam melakukan sebuah pengurutan. Langkah pertama ialah melakukan *pembandingan*. Untuk melakukan pengurutan, dipastikan ada dua buah nilai yang dibandingkan. Pembandingan ini akan menghasilkan bilangan yang lebih besar dari, lebih kecil dari, atau memiliki nilai sama dengan sebuah bilangan lainnya. Langkah kedua ialah melakukan penempatan bilangan setelah melakukan pembandingan. Penempatan bilangan ini dilakukan



setelah didapatkan bilangan lebih besar atau lebih kecil (bergantung pada pengurutan yang digunakan).

Terdapat beberapa teknik (algoritma) untuk melakukan pengurutan seperti *bubble sort*, *insertion sort*, *quick sort*, *merge sort*, dan *selection sort*. Penjelasan untuk setiap teknik sebagai berikut.

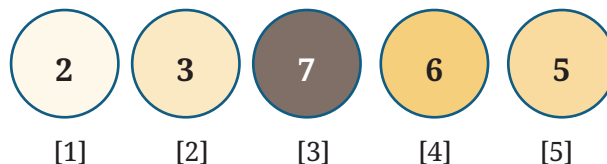
a. **Bubble Sort**

Algoritma *bubble sort* merupakan algoritma yang sudah cukup tua dalam dunia pengurutan. Sesuai namanya, algoritma ini mengilustrasikan bilangan yang diurutkan sebagai sebuah gelembung (*bubble*) yang akan terus naik sampai mencapai tempat atau urutan yang semestinya. *Bubble sort* bekerja dengan menukar elemen yang berdekatan secara berulang hingga mencapai urutan yang semestinya. Jika ada n data bilangan, berikut ialah langkah-langkah yang terdapat pada algoritma *bubble sort* (urut menaik).

1. Bandingkan nilai data ke-1 dan data ke-2. Jika data ke-1 lebih besar daripada data ke-2, tukar posisinya.
2. Ulangi langkah no. 1 pada bilangan berikutnya, yaitu data ke-2 dan ke-3, hingga data ke- $(n-1)$ dan ke- n (jika selesai sampai data ke- n , satu putaran selesai dan nilai data tertinggi akan berada di ujung kanan).
3. Ulangi langkah no. 1 dan 2, untuk n data yang dikurangi 1, yaitu dari $n-1$, $n-2$, dst sampai $n=2$ (sampai habis).

Contoh Eksekusi *Bubble Sort*

Terdapat sebuah deret bilangan seperti berikut: 2, 3, 7, 6, 5. Urutkan bilangan tersebut secara menaik dengan menggunakan algoritma *bubble sort*.



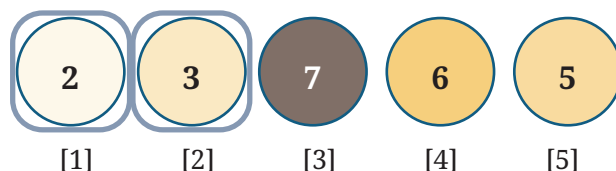
Proses Iterasi (Putaran) Pertama (Empat Kali Perbandingan)

Ambil bilangan pertama dan kedua, lalu bandingkan. Pada kasus ini, bilangan pertama ialah 2 dan bilangan kedua ialah 3. Karena hasil akhir



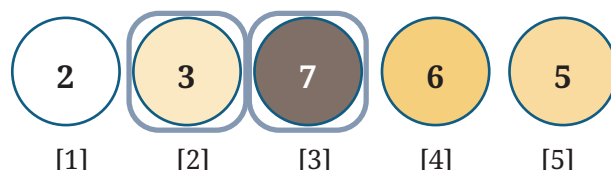
yang diharapkan ialah bilangan yang terurut naik (*ascending*), bilangan yang lebih besar harus berada di sebelah kanan.

Langkah pertama, bandingkan bilangan *pertama* dan *kedua*, yaitu 2 dan 3. Didapatkan 2 lebih kecil dari 3. Maka, urutan bilangan tersebut tetap (2,3).



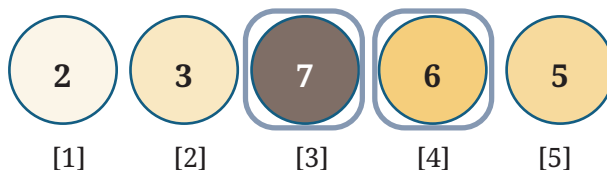
(2, 3, 7, 6, 5) menjadi (2, 3, 7, 6, 5), (tetap, tidak ada pertukaran posisi).

Langkah selanjutnya ialah membandingkan nilai bilangan *kedua* dan *ketiga*, yaitu 3 dan 7. Karena 3 lebih kecil daripada 7, urutan tetap.

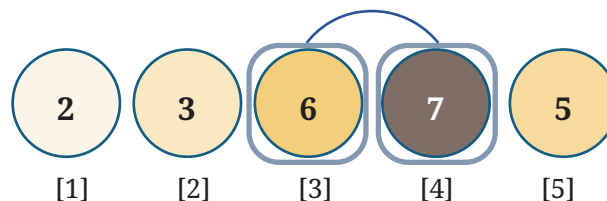


(2, 3, 7, 6, 5) tetap menjadi (2, 3, 7, 6, 5).

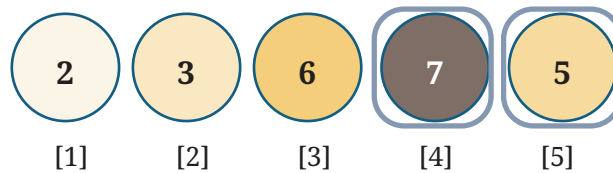
Langkah selanjutnya ialah membandingkan bilangan *ketiga* dan *keempat*. Didapatkan bahwa 7 lebih besar daripada 6. Oleh karena itu, urutan akan ditukar.



(2, 3, 7, 6, 5) menjadi (2, 3, 6, 7, 5).

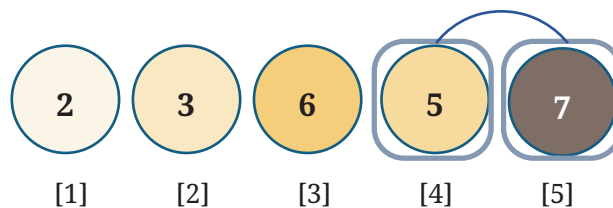


Langkah selanjutnya ialah membandingkan bilangan *keempat* dan *kelima*. Didapatkan fakta bahwa 7 lebih besar daripada 5. Oleh karena itu, urutan akan ditukar.



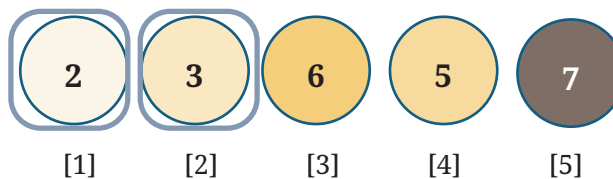
(2, 3, 6, 7, 5) menjadi (2, 3, 6, 5, 7).

Sampai tahap ini, urutan (2, 3, 7, 6, 5) telah menjadi (2, 3, 6, 5, 7). Tentu belum memenuhi pengurutan yang diinginkan. Oleh karena itu, akan dilakukan proses pengurutan kembali mulai dari bilangan pertama, dengan cara seperti di atas. Satu proses perbandingan bilangan pertama sampai dengan bilangan terakhir disebut sebagai satu *iterasi*.

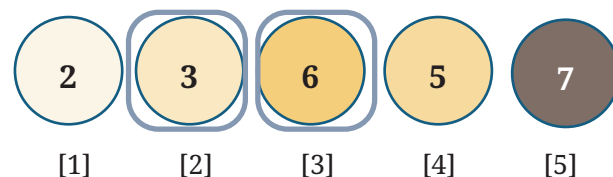


Proses Iterasi Kedua (Tiga Kali Perbandingan)

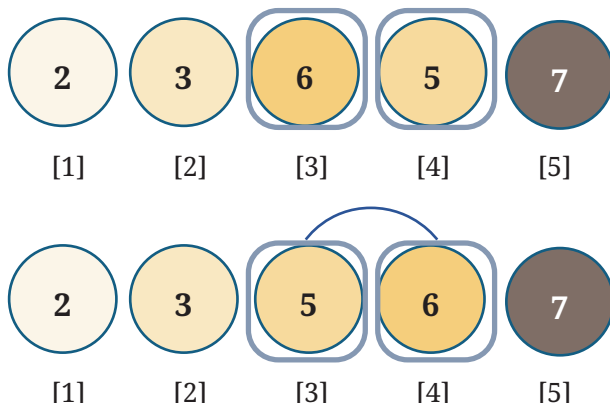
(2, 3, 6, 5, 7) menjadi (2, 3, 6, 5, 7), (tetap, tidak ada pertukaran)



(2, 3, 6, 5, 7) menjadi (2, 3, 6, 5, 7), (tetap tidak ada pertukaran)



(2, 3, 6, 5, 7) menjadi (2, 3, 5, 6, 7), terjadi **pertukaran**.



Proses Iterasi Ketiga (Dua Kali Perbandingan)

(2, 3, 5, 6, 7) menjadi (2, 3, 5, 6, 7), (tetap, tidak ada pertukaran)

(2, 3, 5, 6, 7) menjadi (2, 3, 5, 6, 7), (tetap, tidak ada pertukaran)

Proses Iterasi Keempat (Satu Kali Perbandingan)

(2, 3, 5, 6, 7) menjadi (2, 3, 5, 6, 7), (tetap, tidak ada pertukaran)

Proses pengurutan akan berhenti di tahap ini. Hasil pengurutan secara menaik (*ascending*) sudah dipenuhi. Jika banyak bilangan dan urutan awalnya berbeda, banyak iterasi yang dibutuhkan dapat berbeda.

b. *Insertion Sort*

Algoritma *insertion sort* ialah salah satu algoritma yang digunakan untuk permasalahan pengurutan dalam lis (daftar objek). Sesuai namanya, *insertion sort* mengurutkan sebuah lis dengan cara menyisipkan elemen satu per satu sesuai dengan urutan besar kecilnya elemen hingga semua elemen menjadi lis yang teratur. Misalnya, dalam kasus mengurutkan elemen lis dari yang terkecil hingga terbesar (*ascending*), tahap pertama ialah membaca suatu elemen dengan elemen yang berdekatan. Apabila elemen yang berdekatan dengan elemen saat ini lebih kecil, elemen yang lebih kecil akan ditukar dengan elemen yang lebih besar dan dibandingkan kembali dengan elemen-elemen sebelumnya yang sudah teratur. Apabila elemen saat ini sudah lebih besar daripada elemen sebelumnya, iterasi berhenti. Hal ini dijalankan satu per satu hingga semua lis menjadi teratur.



Berikut ini langkah-langkah algoritma *insertion sort*.

1. Asumsikan ialah nomor iterasi.
2. Ambil bilangan ke- $i+1$.
3. Bandingkan dengan bilangan sebelumnya.
4. Jika bilangan sebelumnya lebih besar, lakukan penukaran, lalu bandingkan lagi dengan bilangan sebelumnya.

Lakukan terus-menerus hingga ditemukan bilangan sebelumnya yang lebih kecil atau hingga sudah sampai di awal lis.

Ilustrasi *Insertion Sort*

Terdapat sebuah deret bilangan seperti berikut: 2, 3, 7, 6, 5 yang direpresentasikan dengan menggunakan kartu. Urutkan bilangan tersebut secara menaik dengan menggunakan algoritma *insertion sort*.

2	3	7	6	5
---	---	---	---	---

Proses Iterasi Pertama

Langkah pertama, bandingkan bilangan *pertama* dan *kedua*, yaitu 2 dan 3. Didapatkan 2 lebih kecil daripada 3. Maka, urutan bilangan tersebut tetap (2,3).

(2, **3**, 7, 6, 5) tetap (2, **3**, 7, 6, 5).

2	3	7	6	5
---	---	---	---	---

Catatan: bilangan yang dicetak tebal ialah bilangan yang telah diperiksa.

Proses Iterasi Kedua

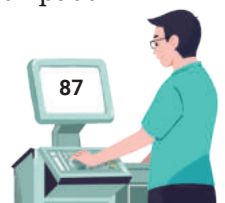
Pada iterasi selanjutnya, ambil bilangan *ketiga*, yaitu 7. Lalu, bandingkan dengan bilangan sebelumnya. Karena 3 lebih kecil daripada 7, urutan tetap.

(2, 3, **7**, 6, 5) tetap (2, 3, **7**, 6, 5).

2	3	7	6	5
---	---	---	---	---

Proses Iterasi Ketiga

Pada iterasi selanjutnya, ambil bilangan *keempat*, yaitu 6. Lalu, bandingkan dengan bilangan sebelumnya. Didapatkan bahwa 7 lebih besar daripada



6. Oleh karena itu, selanjutnya, bandingkan dengan bilangan-bilangan sebelumnya, lalu menukarnya apabila bilangan tersebut lebih besar. Pertama, bandingkan 6 dengan 7. Apakah 6 lebih kecil daripada 7? Karena iya, tukar 6 dengan 7. Lalu, bandingkan lagi dengan bilangan sebelumnya, yaitu 3. Apakah 6 lebih kecil daripada 3? Karena 6 tidak lebih kecil daripada 3, maka 6 sudah berada pada posisi yang benar, yaitu sebelum 7 dan setelah 3.

Proses memindahkan 6 di antara 3 dan 7 ini biasa disebut penyisipan (*insertion*) sehingga nama algoritma ini disebut *insertion sort*.

(2, 3, 7, 6, 5) menjadi (2, 3, 6, 7, 5).

2	3	6	7	5
---	---	---	---	---

Proses Iterasi Keempat

Pada iterasi selanjutnya, kamu mengambil bilangan kelima, yaitu 5. Didapatkan bahwa 7 lebih besar daripada 5. Oleh karena itu, selanjutnya, kamu akan membandingkan dengan bilangan-bilangan sebelumnya, lalu menukarnya apabila bilangan tersebut lebih besar. Pertama, kamu akan membandingkan 5 dengan 6. Apakah 5 lebih kecil daripada 6? Karena iya, kamu akan menukar 5 dengan 6. Setelah itu, kamu akan mengecek dengan bilangan sebelumnya lagi, yaitu 3. Apakah 5 lebih kecil daripada 3? Karena 5 tidak lebih kecil daripada 3, maka 5 sudah pada posisi seharusnya, yaitu setelah 3 dan sebelum 6. Terjadi lagi proses penyisipan kartu 5 di antara 3 dan 6. Akibatnya, urutan berubah dari (2, 3, 6, 7, 5) menjadi (2, 3, 5, 6, 7).

2	3	5	6	7
---	---	---	---	---

Karena semua elemen telah terurut, proses pengurutan berhenti.

c. Selection Sort

Selection sort merupakan algoritma pengurutan yang cukup sederhana. Algoritma ini mencari (menyeleksi) bilangan terkecil/terbesar, bergantung pada urut naik atau turun, dari daftar bilangan yang belum terurut. Kemudian, bilangan tersebut diletakkan dalam daftar bilangan baru yang dijaga keterurutannya. Algoritma ini membagi daftar bilangan menjadi dua bagian, yaitu bagian terurut dan bagian yang belum terurut. Bagian yang terurut di sebelah kiri dan bagian yang belum terurut di sebelah kanan.



Awalnya, semua elemen bilangan dalam daftar ialah bagian yang belum terurut, dan bagian yang terurut kosong.

Berikut ini langkah-langkah yang terdapat pada algoritma *selection sort*.

1. Cari bilangan terkecil yang ada pada bagian belum terurut.
2. Tukar bilangan tersebut dengan bilangan pertama bagian belum terurut, lalu masukkan ke bagian terurut.
3. Ulangi langkah 1 dan 2 sampai bagian yang belum terurut habis.

Ilustrasi urutan *selection sort* dapat dilihat pada tabel berikut.

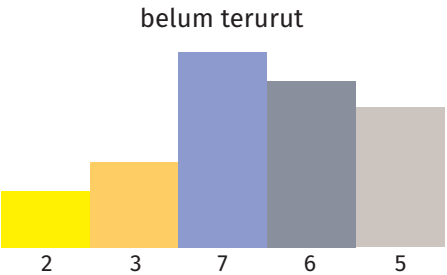
Bagian Terurut	Bagian yang Belum Terurut	Nilai Terkecil dari Bagian Belum Terurut
0	(2,3,7,6,5)	2
(2)	(3,7,6,5)	3
(2,3)	(7,6,5)	5
(2,3,5)	(6,7)	6
(2,3,5,6)	(7)	7
(2,3,5,6,7)	0	

Secara rinci, algoritma *selection sort* yang dikaitkan dengan pemrograman dijelaskan sebagai berikut.

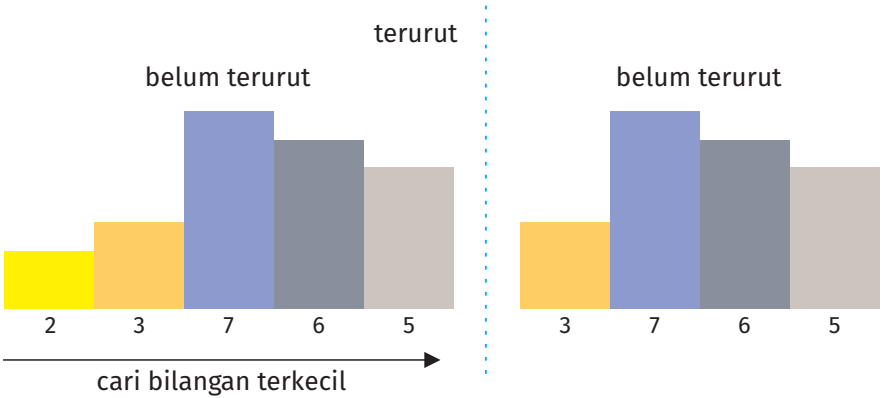
Terdapat sebuah daftar bilangan tidak terurut seperti berikut: 2, 3, 7, 6, 5. Urutkan bilangan tersebut secara menaik dengan menggunakan algoritma *selection sort*.

Proses Iterasi Pertama

Data Awal:



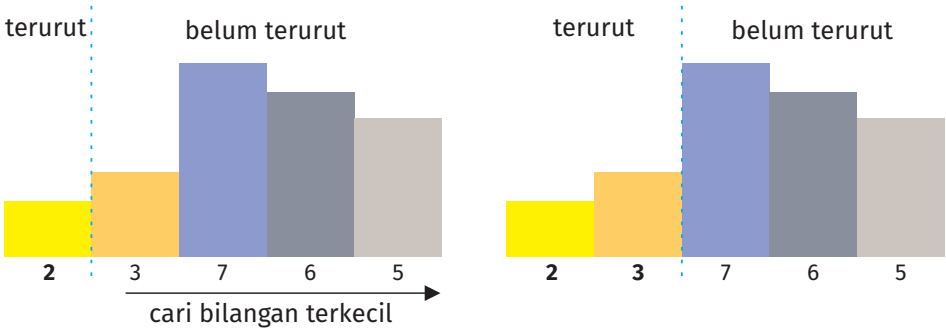
Cari bilangan terkecil di bagian belum terurut: ditemukan 2 sebagai bilangan terkecil.



Tukar bilangan 2 dengan bilangan pertama bagian belum terurut. Geser batas bagian yang sudah terurut ke kanan sehingga 2 menjadi bagian yang sudah terurut. Dalam ilustrasi ini, angka yang dicetak tebal menunjukkan bilangan yang sudah terurut.

Proses Iterasi Kedua

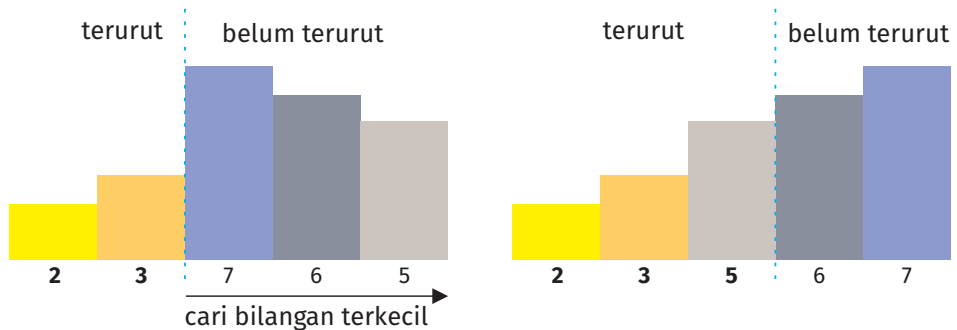
Cari bilangan terkecil di bagian belum terurut, ditemukan angka 3 sebagai bilangan terkecil.



Tukar bilangan 3 dengan bilangan pertama bagian belum terurut. Geser batas bagian yang sudah terurut ke kanan sehingga 3 menjadi bagian yang sudah terurut.

Proses Iterasi Ketiga

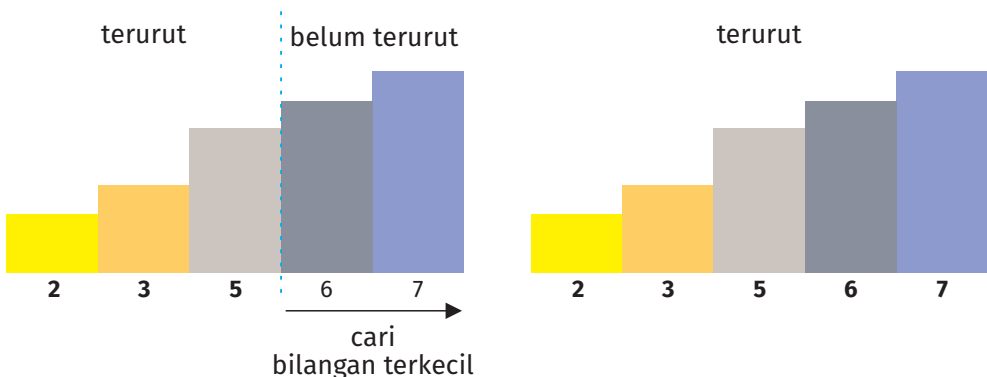
Cari bilangan terkecil di bagian belum terurut, ditemukan angka 5 sebagai bilangan terkecil.



Tukar bilangan 5 dengan bilangan pertama bagian belum terurut, yaitu 7. Geser batas bagian yang sudah terurut ke kanan sehingga 5 menjadi bagian yang sudah terurut.

Proses Iterasi Keempat

Cari bilangan terkecil di bagian belum terurut, ditemukan angka 6 sebagai bilangan terkecil. Tukar bilangan 6 dengan bilangan pertama bagian belum terurut. Di bagian akhir karena data tinggal dua, setelah proses penukaran, algoritma telah selesai dilaksanakan.





Aktivitas 10.2-09-U

Mencari dan Mengurutkan

Identifikasilah hal-hal di kehidupan kamu yang melibatkan proses mencari, proses mengurutkan, atau keduanya sekaligus. Algoritma pengurutan apakah yang selama ini kamu gunakan?



Aktivitas 10.2-10-UP

Tebak Angka

Untuk memahami masalah pencarian, kamu akan bermain tebak angka. Pada saat bermain, pahami permainan tersebut dan identifikasi aspek-aspek masalah pencarian pada permainan tersebut. Carilah strategi terbaik untuk menemukan angka yang dimiliki oleh temanmu dengan jumlah pengecekan sesedikit mungkin.

Skenario Permainan

Pada permainan ini, kamu harus berpasangan dengan salah seorang teman. Temanmu akan memilih sebuah angka bilangan bulat antara 1 – 100 (inklusif, angka 1 dan 100 juga boleh dipilih), dan angka tersebut akan dia rahasiakan. Tugas kamu ialah menemukan angka tersebut.

Untuk menemukan angka tersebut, kamu harus mengecek apakah angka tebakanmu ialah angka yang dimiliki oleh temanmu. Kamu hanya dapat mengecek angka satu per satu dengan menyebutkan angka tebakan kamu tersebut.

Setiap kali kamu menebak, temanmu harus menjawab satu dari tiga kemungkinan berikut.

- “Benar”, apabila angka yang kamu tebak sama dengan angka yang dimiliki temanmu.
- “Angka milikku lebih kecil”, apabila angka yang dimiliki temanmu lebih kecil dari tebakanmu.
- “Angka milikku lebih besar”, apabila angka yang dimiliki temanmu lebih besar dari tebakanmu.



Tentu saja, kamu dapat menebak angka apa pun. Namun, cari strategi yang membuat kamu dapat dengan cepat (atau dengan kata lain jumlah tebakan sesedikit mungkin) menemukan angka yang dipilih oleh temanmu.

Catatlah angka-angka yang kamu tebak dan jumlah tebakan yang kalian lakukan di lembar kerja yang disediakan. Lakukan permainan ini minimal sebanyak dua kali. Pada permainan berikutnya, kalian dapat bertukar peran.

Ilustrasi

Berikut ini ilustrasi dari permainan di atas. Pada permainan ini, Andi dan Binti bermain secara berpasangan. Andi memilih angka, sedangkan Binti harus menebak angka tersebut.

Mula-mula, Andi memilih sebuah angka antara 1 s.d. 100 (inklusif) dan

Binti mencatat tebakannya di lembar kerjanya

Binti: “50?”

Andi: “Angka milikku lebih kecil.”

Binti: “30?”

Andi: “Angka milikku lebih besar.”

Binti: “40?”

Andi: “Angka milikku lebih kecil.”

Binti: “35?”

Andi: “Benar!”

Pada percakapan di atas, terlihat bahwa Binti dapat menebak angka yang dipilih Andi dalam empat kali tebakan. Percakapan tersebut dicatat dalam lembar kerja, pada permainan ke-0.

Kamu dapat memanfaatkan tabel berikut yang digunakan untuk mencatat angka yang ditebak dan berapa kali penebakan dilakukan. Pada permainan ke-0, diberikan contoh isian dari ilustrasi yang diberikan di atas. Terlihat bahwa Andi memilih angka 49. Binti memerlukan sebanyak 4 kali penebakan untuk menebak dengan benar angka yang dipilih oleh Andi, yaitu menebak secara berurutan 50, 40, 30, dan 35 .



Jawabannya tentu bergantung pada tebakan berikutnya.

Siapa Andi?

Siapa Binti?

Permainan ke-	Tebakan ke-																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
0	50	40	30	35														
1																		
2																		
3																		

Apa yang kamu diskusikan?

Setelah bermain, saatnya kamu memikirkan makna permainan tersebut, dan cara kamu bermain. Beberapa poin diskusi yang akan kamu lakukan ialah seperti berikut.

1. Apakah permainan ini merupakan masalah pencarian?
2. Apabila Binti menjalankan strategi yang tepat, berapa kali jumlah maksimal tebakan yang benar-benar dia perlukan?
3. Strategi pencarian seperti apa yang kamu lakukan untuk menebak sesedikit mungkin?
4. Apakah strategimu berbeda dengan strategi yang dilakukan temanmu? Jika berbeda, apa perbedaannya?
5. Strategi paling bagus apa yang dapat kalian temukan untuk menemukan angka dengan jumlah tebakan paling sedikit?
6. Adakah cara lain untuk “mencari” angka yang ditebak?

Apa yang kamu lakukan?

Tuliskan algoritma Tebak Angka dalam bahasa Indonesia. Masukkan dalam Buku Kerja Siswa.

Tantangan

Simpan tantangan ini sampai kamu sudah dapat memprogram dari modul Pemrograman. Jika kamu terpaksa melakukan permainan tebak angka sendiri karena belajar di rumah, ayo, lakukan hal ini bersama sebuah program komputer! Program komputer akan menjadi Andi, angka yang dipikirkan akan ditentukan secara acak oleh komputer. Kamu dapat membuat program komputer yang



menjadi Andi jika kamu sudah belajar modul pemrograman. Catat hal ini sebagai salah satu hal yang akan kamu lakukan!



Aktivitas 10.2-11-UP Bermain Kartu

Skenario Permainan

1. Untuk aktivitas ini, kamu memerlukan 10 kartu yang masing-masing bertuliskan angka 1 sampai 10.
2. Aktivitas dapat dilakukan secara mandiri atau berkelompok.
3. Kamu akan diberikan sebuah kartu bertuliskan angka dari 1 - 10.
4. Kesepuluh kartu tersebut kamu kocok dan letakkan dalam bentuk barisan di atas meja. Kartu diletakkan tertutup.
5. Kamu harus dapat mengurutkan semua kartu secara menaik. Kartu yang berada di paling kiri barisan harus yang paling kecil.
6. Untuk mengurutkan, kamu harus melakukan serangkaian pertukaran kartu. Pertukaran dilakukan dengan membuka dua buah kartu. Apabila diperlukan, kamu dapat menukar posisi kedua kartu tersebut.
7. Kamu diminta untuk menyusun algoritma pertukaran yang dapat dilakukan untuk memastikan semua kartu dalam posisi terurut. Kamu dapat memilih untuk menggunakan salah satu dari tiga algoritma pengurutan yang disampaikan pada bagian konsep.

Apa yang kamu diskusikan?

Setelah bermain, saatnya memikirkan permainan tersebut dan cara kamu bermain. Berikut ini beberapa poin yang penting untuk didiskusikan.

1. Apakah permainan tadi merupakan masalah pengurutan?
2. Strategi pengurutan seperti apa yang kamu lakukan untuk melakukan pengecekan dan pertukaran sesedikit mungkin?
3. Apakah strategi kamu berbeda dengan strategi yang dilakukan oleh temanmu? Jika berbeda, apa perbedaannya?



4. Strategi paling bagus apa yang dapat kamu temukan untuk mengurutkan dengan banyaknya pertukaran paling sedikit?
5. Adakah kondisi yang membuat kamu melakukan banyak sekali pertukaran untuk mengurutkan kartu secara menaik?

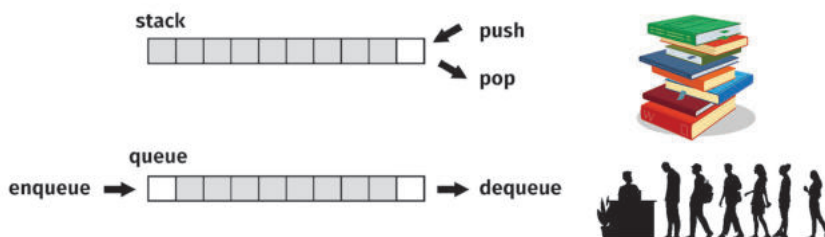
Ayo, Renungkan!

1. Apakah kamu sudah pernah melakukan permainan ini?
2. Saat mengurutkan kartu, apakah kamu senang?
3. Apakah kamu paham bahwa mengurutkan kartu itu suatu proses pengurutan?
4. Apakah kamu berhasil menemukan cara yang paling cepat untuk mengurutkan kartu tersebut?
5. Apakah kamu merasa ada masalah lain yang serupa dengan permainan tadi?
6. Pelajaran paling berkesan apa yang kamu dapatkan dari permainan ini?

Jawablah pertanyaan tersebut dalam Lembar Refleksi pada Buku Catatan, dan jangan lupa mencatat kegiatan dalam Jurnal.

D. Memilih Struktur Data untuk Masalah di Kehidupan Nyata

Berikut ini kamu akan belajar tentang hal-hal yang kamu temukan dalam keseharian, misalnya tentang tumpukan dan antrean.



Gambar 2.3 Ilustrasi struktur data antrean serta tumpukan dan contohnya di dunia nyata.

Kita akan mempelajari dua buah konsep cara penyimpanan data/objek dalam sebuah struktur yang akan menentukan urutan pemrosesan data/objek tersebut, yaitu *stack* (tumpukan) dan *queue* (antrean). Kedua konsep ini memiliki prosedur yang berbeda dalam menyimpan dan mengeluarkan data. Kedua konsep tersebut masing-masing memiliki peranan yang berbeda dan digunakan pada situasi yang berbeda pula.

Bayangkan sebuah loket di sebuah rumah sakit. Para pasien yang akan berobat diminta untuk mendaftar lebih dahulu di loket penerimaan dan mengisi formulir pendaftaran. Setelah formulir tersebut diisi, para pasien akan mengembalikan formulir ke loket dan menunggu dipanggil oleh petugas. Kebetulan, di pagi hari, dokter yang bertugas belum datang sehingga para pasien harus menunggu. Ketika sang dokter tiba, petugas loket akan memanggil para pasien satu per satu untuk mendapat layanan.

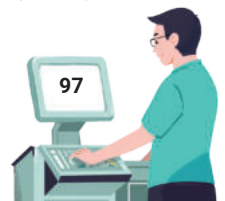
Perhatikan bagaimana urutan pasien itu dipanggil oleh petugas loket.

- Misalkan, petugas loket menumpuk formulir-formulir tersebut. Formulir yang baru diterima diletakkan *di atas* formulir yang sudah diterima sebelumnya. Kemudian, ketika memanggil pasien, petugas tersebut memanggil dengan urutan mulai dari formulir yang berada *di atas* tumpukan. Menurut kamu apakah urutan tersebut adil/sesuai dengan yang diharapkan para pasien? Mengapa?
- Bagaimana cara petugas menyusun tumpukan formulir dan/atau cara urutan memanggil para pasien dari tumpukan formulir sedemikian rupa sehingga pasien yang datang dan mengisi formulir lebih dulu, akan *dipanggil lebih dulu* juga (dan sebaliknya)?

Dalam dunia komputasi/informatika, terkadang, kamu perlu untuk menyimpan data/objek dalam suatu urutan tertentu, untuk kemudian/sewaktu-waktu diambil/dikeluarkan kembali, mungkin untuk diproses lebih lanjut atau untuk tujuan-tujuan lain. Ada dua cara utama kamu dapat melakukan penyimpanan ini.

1. Antrean (*Queue*)

Pada metode ini, objek-objek disimpan dalam metode penyimpanan yang berupa sebuah antrean sehingga objek yang *pertama/lebih dahulu datang*, juga akan *lebih dahulu keluar/selesai*, layaknya sebuah antrean di loket, pintu masuk, dll. Prinsip ini disebut juga sebagai prinsip *First In First Out (FIFO)*.



Dalam sebuah antrean orang, misalnya, jelas orang yang pertama datang akan berada di depan antrean, dan harus menjadi yang pertama yang mendapat pelayanan.

2. Tumpukan (*Stack*)

Pada metode ini, objek-objek disimpan dalam metode penyimpanan yang menyerupai sebuah tumpukan (misal: tumpukan piring). Dengan demikian, objek yang *pertama/lebih dahulu disimpan*, justru akan menjadi yang terakhir keluar. Prinsip ini disebut juga *Last In First Out (LIFO)*. Dalam tumpukan piring, misalnya, piring pertama yang diletakkan, akan berada di posisi paling bawah, dan jika kamu ambil piring satu per satu dari tumpukan itu, tentunya piring yang berada di posisi paling bawah tersebut akan menjadi yang terakhir diambil.

Baik dalam kehidupan sehari-hari maupun dalam dunia informatika, kedua konsep urutan penyimpanan data tersebut memiliki peran dan kegunaan masing-masing. Ada permasalahan-permasalahan/situasi di mana antrean (FIFO) lebih cocok digunakan, dan sebaliknya ada juga permasalahan-permasalahan di mana tumpukan (LIFO) lebih tepat diterapkan. Untuk lebih memahami kedua konsep ini dan bagaimana mereka digunakan, mari, kamu lakukan beberapa aktivitas di bawah ini.



Aktivitas 10.2-12-U

Penggunaan *Stack* dan *Queue* secara Tepat

Pada aktivitas ini, kamu akan membaca beberapa skenario kondisi, baik dalam dunia sehari-hari maupun dalam dunia informatika. Tugas kamu ialah memikirkan, pada setiap kondisi/skenario tersebut, manakah yang lebih tepat digunakan/lebih relevan menggambarkan situasi tersebut, apakah *stack* ataukah *queue*. Berikan penjelasan mengapa kamu memilih jawaban tersebut!

1. Di persimpangan jalan, terdapat lampu merah. Apabila lampu merah menyala, mobil-mobil yang datang ke persimpangan tersebut harus berhenti dulu. Ketika lampu berubah menjadi hijau, semua mobil perlahan-lahan berjalan kembali dalam urutan tertentu. Manakah yang lebih tepat menggambarkan situasi tersebut?



2. Ketika menjelajah web/internet, kamu menggunakan sebuah *browser* (misal Firefox, Chrome dll). Terdapat sebuah fitur yang memungkinkan kamu untuk bergerak dari satu halaman yang sudah kamu kunjungi ke halaman lainnya, yaitu dengan menekan tombol *Back* dan *Forward*. Misalnya, kamu mengunjungi halaman A, kemudian B, lalu C. Jika kamu kemudian menekan tombol *Back*, dari halaman C, kamu akan kembali ke halaman B. Jika kamu tekan lagi tombol *Back* (pada saat ada di B), kamu akan kembali ke A. Jika kemudian kamu tekan tombol *Forward*, kamu akan kembali halaman B. Jika kamu tekan sekali lagi tombol *Forward*, kamu akan kembali ke halaman C. Oleh karena itu, aplikasi *browser* tersebut harus menyimpan (dan mengingat) semua halaman yang sudah pernah kamu kunjungi sebelumnya (biasa disebut sebagai Riwayat atau *History*). Bentuk penyimpanan yang manakah (*stack* atau *queue*) yang paling tepat digunakan untuk menyimpan Riwayat pada *browser*?



3. Mesin *printer* bertugas untuk mencetak dokumen yang dikirimkan dari sebuah komputer. Satu buah *printer* dapat terhubung ke beberapa buah komputer sekaligus, dan semuanya dapat mengirim perintah kepada *printer* tersebut untuk mencetak dokumen yang berbeda-beda. *Printer* tersebut tentunya hanya dapat mencetak satu buah dokumen dalam satu waktu tertentu, dan mungkin membutuhkan beberapa detik/menit untuk menyelesaikan proses cetak satu dokumen. Oleh karena itu, ketika *printer* sedang sibuk mencetak sebuah dokumen dari sebuah komputer, kemudian datang permintaan mencetak dari beberapa komputer yang lain (yang berbeda), *printer* tersebut harus menyimpan dokumen-dokumen yang baru datang tersebut agar nanti dapat dicetak ketika proses pencetakan yang sedang berjalan saat ini sudah selesai. Manakah yang lebih tepat digunakan, *stack* atau *queue* untuk penyimpanan dokumen-dokumen yang sedang “menunggu giliran” untuk dicetak tadi?



4. Pada sebuah aplikasi pengolah dokumen, biasanya terdapat fasilitas untuk melakukan *Undo* dan *Redo*. Operasi *Undo* akan membatalkan langkah/tindakan terakhir yang kamu lakukan saat mengedit dokumen (misal, jika kamu menyadari ada kesalahan pada langkah terakhir kamu), sedangkan *Redo* digunakan untuk mengulang kembali operasi yang baru saja dibatalkan dengan sebuah *Undo*. Proses *Undo* dan *Redo* ini dapat dilakukan sampai dengan operasi pertama setelah sebuah dokumen dibuka/disimpan. Misalnya, terjadi rangkaian kejadian berikut.
- Budi membuka dokumen A.
 - Budi menambahkan judul pada dokumen A.
 - Budi menulis sebuah paragraf pada dokumen A.
 - Budi menambahkan sebuah tabel pada dokumen A.
 - Budi menyisipkan sebuah gambar pada dokumen A.

Apabila kemudian Budi menekan tombol *Undo*, operasi terakhir (yaitu penambahan gambar) akan dibatalkan sehingga gambar tersebut akan hilang dari dokumen. Jika kemudian Budi menekan tombol *Undo* sekali lagi, operasi terakhir sebelum itu (yaitu menambahkan tabel) juga akan dibatalkan sehingga tabel tersebut akan hilang dari dokumen. Jika kemudian Budi menekan tombol *Redo*, operasi *Undo* yang terakhir (yaitu yang menghilangkan tabel) akan dibatalkan sehingga tabel tersebut akan muncul kembali.

Jelas bahwa aplikasi perlu untuk menyimpan data-data berupa tindakan/operasi apa saja yang dilakukan oleh pengguna dari awal sampai akhir, serta efeknya terhadap dokumen, agar dapat memberikan fungsionalitas *Undo* dan *Redo* tersebut. Manakah di antara *stack* dan *queue* yang lebih tepat digunakan untuk menyimpan operasi-operasi tersebut?



Lembar Kerja Siswa

Untuk setiap kasus di atas, lakukan analisis penggunaan *stack* dan *queue* dengan mengisi LKS ini.

Persoalan	Stack	Queue	Saya Pilih.....
Persimpangan lampu merah			
Penjelajahan Internet			
Antrean permintaan <i>print</i> dokumen dalam sebuah komputer			
<i>Undo Redo</i>			



Aktivitas 10.2-13-UP Simulasi *Stack*

Pada aktivitas ini, kamu akan bermain dengan satu orang siswa lainnya. Satu orang harus berperan menjadi *Pemberi Perintah* dan satu lagi harus berperan sebagai *Simulator*. Permainan dimulai dengan Pemberi Perintah memberikan sebuah perintah simulasi (yang akan dijelaskan di bawah), dan kemudian Simulator harus menjalankan simulasi dan memberikan jawaban yang benar. Jawaban tersebut kemudian harus diperiksa oleh Pemberi Perintah dan kemudian harus dinyatakan apakah jawaban tersebut benar atau salah. Setelah itu, kedua orang bertukar peran: Simulator harus menjadi Pemberi Perintah dan Pemberi Perintah menjadi Simulator. Lakukan pertukaran ini sampai beberapa kali. Orang yang berhasil mendapatkan jawaban benar sebanyak mungkin akan menjadi pemenangnya.

Berikut ini format/bentuk perintah dan bentuk jawaban yang diinginkan. Kita asumsikan ada sebuah *stack* yang mampu menyimpan nilai-nilai bilangan. Setiap perintah simulasi berisi kumpulan dari 2 buah perintah:

- PUSH X
- POP



Perintah PUSH digunakan untuk menyimpan nilai ke dalam *stack*. Perintah ini harus diikuti oleh sebuah bilangan bulat X yang akan disimpan ke dalam *stack*. Perintah POP digunakan untuk mengeluarkan angka yang *berada di atas tumpukan* saat ini. Jika saat ini tumpukan kosong, perintah POP tidak memberikan efek apa-apa. Berikut ini contoh sebuah perintah simulasi dan hasilnya.

Perintah	Isi Stack	Hasil POP
PUSH 5	5	
PUSH 3	5,3	
PUSH 2	5,3,2	
PUSH 4	5,3,2,4	
POP	5,3,2	4
PUSH 6	5,3,2,6	
POP	5,3,2	6
POP	5,3	2
POP	5	3

Ketika seorang Simulator menerima sebuah perintah simulasi, dia harus memberikan jawaban berupa daftar *bilangan yang akan dikeluarkan* dari *stack*, sesuai dengan urutan perintah simulasi yang dia terima. Misalnya, pada contoh di atas, Simulator harus memberikan jawaban berupa:

4
6
2
3

Tentunya, banyaknya angka pada jawaban harus sama dengan banyaknya perintah POP yang diberikan oleh Simulator.



Lembar Kerja Siswa

Untuk permainan peran ini, dapat dipakai LKS berikut ini.

Pemberi Perintah	Catatan Simulator	Isi <i>Stack</i>	Hasil POP

Aktivitas 10.2-14-UP Simulasi *Queue*



Pada aktivitas ini, kamu akan melakukan simulasi operasi pada sebuah *queue*. Serupa dengan aktivitas sebelumnya, aktivitas ini dijalankan oleh dua orang yang akan bertugas sebagai Pemberi Perintah dan Simulator. Pemberi Perintah akan memberikan kumpulan perintah yang berisi operasi pada *queue*, sedangkan Simulator harus memberikan jawaban berupa *rangkaian isi queue* yang dihasilkan dari setiap perintah yang diberikan.

Format perintah sebagai berikut.

- ENQUEUE X: memasukkan sebuah bilangan bulat ke dalam *queue*.
- DEQUEUE: membuang/mengeluarkan bilangan yang berada pada posisi pertama antre.



Untuk setiap perintah, Simulator harus menuliskan *apa isi queue* setiap kali perintah tersebut selesai dijalankan.

Sebagai contoh pemberi perintah memberikan perintah-perintah sebagai berikut.

Perintah	Simulator Menulis Isi Queue Setelah Setiap Perintah Dijalankan	Hasil DEQUEUE
ENQUEUE 5	5	
ENQUEUE 3	5, 3	
DEQUEUE	3	5
ENQUEUE 4	3, 4	
DEQUEUE	4	3

Jika Simulator harus memberikan 5 baris jawaban berupa isi dari *queue* setelah setiap perintah dijalankan, hasilnya:

1. 5
2. 5, 3
3. 3
4. 3, 4
5. 4

Lembar Kerja Siswa

Untuk permainan peran ini, dapat dipakai LKS.

Pemberi Perintah	Catatan Simulator	Isi Queue	Hasil Enqueue



Ayo, Renungkan!

1. Apakah kamu dapat memahami dengan baik perbedaan dari konsep *stack* dan *queue*?
2. Jika diberikan sebuah kondisi di dunia nyata/informatika, dapatkan kamu menentukan apakah *stack* atau *queue* yang lebih relevan diterapkan sebagai metode penyimpanan?
3. Dapatkah kamu mencari contoh-contoh lain penerapan *stack* dan *queue* dalam kehidupan sehari-hari?
4. Apakah kamu dapat memainkan permainan simulasi *stack* dan *queue* di atas dengan baik? Apakah permainan tersebut membantu proses pemahaman kamu terhadap kedua konsep tersebut?



Aktivitas 10.2-15-UP Studi Kasus: Survei Desa

Perhatikan Problem 2 pada Aktivitas 10.2-06-UP: Latihan Struktur Perulangan. Pada problem tersebut, kamu hanya diminta untuk menghitung rata-rata dari maksimum 1.000 nilai. Sekarang, misalkan kamu akan memodifikasi program tersebut untuk mensurvei usia dari warga suatu desa. Tugas kamu tidak hanya sekadar mencetak rata-rata dari usia tersebut, tetapi juga mencetak nilai tengah (median) dari usia penduduk di desa tersebut. Menurut kamu, perbedaan paling dasar apa yang akan terjadi di program yang kamu buat?

Rancanglah program tersebut, baik dalam bentuk diagram alir, pseudokode, atau kode program yang untuk mencapai tujuan di atas. Untuk bentuk diagram alir atau pseudokode, kamu dapat menggunakan blok program *sort()* untuk mengurutkan data. Jika menggunakan kode program, kamu dapat menggunakan fungsi pengurutan bawaan bahasa pemrograman yang digunakan.



Pilihlah satu jawaban yang tepat.

1. Mengapa suatu algoritma perlu direpresentasikan dalam berbagai bentuk seperti bentuk naratif, diagram alir, atau pseudokode?
 - A. Untuk Mengubah algoritma menjadi program.
 - B. Menyusun algoritma dengan menggunakan simbol matematika.
 - C. Menggambarkan algoritma agar dapat dipahami oleh orang lain.
 - D. Menyederhanakan algoritma menjadi lebih pendek.
 - E. Membuatnya dapat dieksekusi oleh komputer.
2. Diagram alir dapat digunakan untuk merepresentasikan atau menggambarkan sebuah algoritma karena
 - A. lebih efisien daripada pseudokode
 - B. lebih mudah dipahami oleh mesin
 - C. memerlukan lebih sedikit langkah
 - D. memungkinkan sebuah algoritma divisualisasikan
 - E. mengasah aspek kreativitas
3. Benar atau salah pernyataan berikut:
pseudokode dapat dijalankan oleh komputer karena telah ditulis dengan menggunakan bahasa pemrograman.
4. Apa yang dimaksud dengan empat elemen generik dalam bahasa pemrograman prosedural?
 - A. Variabel, ekspresi, struktur kontrol percabangan, dan struktur kontrol perulangan.
 - B. Kata kunci, pernyataan, operator, dan angka.
 - C. Fungsi, kelas, objek, dan metode.



- D. Tipe data, pengulangan, kondisi, dan perbandingan.
 - E. Perkalian, pembagian, penjumlahan, dan pengurangan.
5. Mengapa pemahaman empat elemen generik dalam bahasa pemrograman prosedural penting dalam pembuatan program?
- A. Merupakan bagian utama dari bahasa pemrograman.
 - B. Hanya berlaku dalam bahasa pemrograman.
 - C. Digunakan dalam semua bahasa pemrograman.
 - D. Tidak memengaruhi pembuatan program.
 - E. Merupakan sebuah ketentuan baku dalam pemrograman.
6. Saat kamu merancang suatu algoritma untuk menyelesaikan suatu permasalahan, kegiatan apa yang akan kamu lakukan?
- A. Membuat program segera tanpa menentukan tujuan atau format masukan/keluaran.
 - B. Membaca teori pemrograman terlebih dahulu.
 - C. Menentukan tujuan program, format masukan, dan format keluaran terlebih dahulu.
 - D. Membaca semua aktivitas di buku sebelum membuat program.
 - E. Mencari kode program serupa di Internet.
7. Apa yang dimaksud dengan variabel dalam bahasa pemrograman?
- A. Sebuah perintah untuk menghentikan eksekusi program.
 - B. Sebuah struktur kontrol untuk melakukan iterasi.
 - C. Nama dari suatu lokasi memori yang digunakan untuk menyimpan data.
 - D. Sebuah tipe data untuk menyimpan nilai karakter.
 - E. Memori komputer.



8. Apa yang dimaksud dengan struktur kontrol keputusan dalam bahasa C?
- A. Sebuah tipe data yang digunakan untuk menyimpan nilai logika.
 - B. Sebuah pernyataan yang digunakan untuk menginisialisasi variabel.
 - C. Sebuah struktur yang digunakan untuk mengatur alur program berdasarkan kondisi tertentu.
 - D. Sebuah tipe data untuk menyimpan angka desimal.
 - E. Suatu struktur data yang memungkinkan penyimpanan data yang bercabang atau bersifat hirarkis.
9. Kamu ingin mencari nomor telepon temanmu. Namun, kamu tidak tahu di mana kamu menyimpannya. Kamu memutuskan untuk mencari nomor tersebut satu per satu dari awal buku telepon. Teknik pencarian ini mirip dengan
- A. *bubble sort*
 - B. *quick Sort*
 - C. *linear/sequential search*
 - D. *binary search*
 - E. *merge sort*
10. Kamu memiliki daftar angka dalam urutan acak dan ingin mengurutkannya dari angka terkecil ke angka terbesar dengan memindahkan elemen-elemen yang lebih kecil ke posisi terdepan daftar. Teknik pengurutan ini mirip dengan algoritma
- A. *bubble sort*
 - B. *quick Sort*
 - C. *insertion search*
 - D. *selection search*
 - E. *merge sort*



11. Ketika kamu mengantre di kasir supermarket, konsep apa di dalam komputer, yang paling mirip dengan cara antrean ini?
- A. *stack*
 - B. *queue*
 - C. *array*
 - D. *linked list*
 - E. *sorting*
12. Pada program pengolah kata, aksi *undo* dan *redo* menggambarkan struktur data
- A. *stack*
 - B. *queue*
 - C. *array*
 - D. *linked list*
 - E. *tree*



Pengayaan

Jika ingin belajar lebih mendalam tentang materi di atas, kamu dapat mengunjungi tautan berikut ini.

PENCARIAN (*SEARCHING*)

- Search Algorithm: <https://buku.kemdikbud.go.id/s/cmxxkmb>
- Binary Search: <https://buku.kemdikbud.go.id/s/3dyxlp>

PENGURUTAN (*SORTING*)

- Sorting Algorithm: <https://buku.kemdikbud.go.id/s/fkijda>
- <https://visualgo.net/en>

